

M2 File Format Specification

Format: Blizzard M2 Model Format (World of Warcraft)

Byte Order: Little-endian

Primary Reference: <https://wowdev.wiki/M2>

Implementation: WhiteoutLib `whiteout::m2` — parser + writer for `.m2`, `.skin`, `.anim`, `.bone`, `.skel`

Table of Contents

- [M2 File Format Specification](#)
 - [Table of Contents](#)
 - [1. Overview](#)
 - [2. Format Variants](#)
 - [2.1 Classic \(MD20\)](#)
 - [2.2 Chunked \(MD21\)](#)
 - [3. Version History](#)
 - [4. Primitive Types](#)
 - [Scalar Types](#)
 - [Normalized Integers](#)
 - [Vector / Matrix Types](#)
 - [Compressed Quaternion \(CompatQuaternion\)](#)
 - [Extent \(Bounding Box + Sphere\)](#)
 - [5. Core Patterns](#)
 - [5.1 M2Array](#)
 - [5.2 Animation Tracks](#)
 - [5.3 Interpolation](#)
 - [5.4 Global Sequences](#)
 - [5.5 ParticleAnimationTrack](#)
 - [6. MD20 Header](#)
 - [6.1 Binary Layout](#)
 - [6.2 Global Flags](#)
 - [7. Skeleton and Animation](#)
 - [7.1 Global Sequence Loops](#)
 - [7.2 Animation Sequences](#)
 - [7.3 Sequence Flags](#)
 - [7.4 Sequence Index Hash \(Animation Lookup\)](#)
 - [7.5 Bones](#)
 - [7.6 Bone Flags](#)
 - [7.7 Key-Bone Lookup](#)
 - [8. Geometry and Rendering](#)
 - [8.1 Vertices](#)
 - [8.2 Textures](#)
 - [8.3 Texture Types](#)
 - [8.4 Materials \(Render Flags\)](#)

- 8.5 Colors and Transparency
- 8.6 Texture Transforms
- 8.7 Lookup / Combo Tables
- 9. Bounding and Collision Volumes
- 10. Scene Objects
 - 10.1 Attachments
 - 10.2 Events
 - 10.3 Lights
 - 10.4 Cameras
- 11. Effects
 - 11.1 Ribbon Emitters
 - 11.2 Particle Emitters
 - 11.3 Particle Flags
- 12. SKIN File (`.skin`)
 - 12.1 Skin Profile
 - `lodVertexBase` — HD Model Vertex Splitting
 - 12.2 Skin Sections (Submeshes)
 - 12.3 Batches (Texture Units)
 - 12.4 Shadow Batches
 - 12.5 HD Model Identification
- 13. Chunked M2 (MD21 + Chunks)
 - 13.1 Chunk Format
 - 13.2 Chunk Catalogue
 - 13.3 File ID Reference Chunks
 - 13.4 Particle Extension Chunks
 - 13.5 Parent Data Chunks
 - 13.6 LOD and Rendering Chunks
 - 13.7 Waterfall / PBR Chunks
 - 13.8 Collision and Housing Chunks
 - 13.9 Miscellaneous Chunks
- 14. Skeleton File (`.skeleton`)
- 15. Bone File (`.bone`)
- 16. Animation File (`.anim`)
 - Pre-Legion (raw)
 - Legion+ (chunked)
- 17. Physics File (`.phys`) and PFDC
 - 17.1 File Structure Overview
 - 17.2 Version History
 - 17.3 PHYS Header
 - 17.4 PHYT — Physics Type
 - 17.5 Body Structures
 - 17.6 Shape Structures
 - 17.7 Shape Data: CAPS (Capsule)
 - 17.8 Shape Data: PLYT (Polytope) — CORRECTED
 - 17.9 Joint Structure
 - 17.10 Joint Types

- SHOJ (Shoulder Joint) — 116 bytes
- SHJ2 (Shoulder Joint v2) — 124 bytes
- WLJ2 (Weld Joint v2) — 112 bytes
- WLJ3 (Weld Joint v3) — 116 bytes
- REVJ (Revolute Joint) — 112 bytes
- REV2 (Revolute Joint v2) — 120 bytes
- 17.11 PFDC Inline Physics
- 17.12 How a model finds its physics
- 17.13 Where the field names come from
- 18. Chunked Geometry and Model3 — 11.x Client Formats
 - 18.1 Chunked Geometry (`.skin` successor)
 - 18.2 Geometry Collision Chunks
 - 18.3 Model3 (`.m3`) — WoW
- 19. Bundle Naming Conventions
- 20. WhiteoutLib Implementation Notes
 - 20.1 Parser
 - 20.2 Writer
 - 20.3 API Surface
 - 20.4 Known Limitations
- 21. Corpus Validation Results
 - 21.1 Overview
 - 21.2 Key Findings
 - 21.3 Size Validation
 - 21.4 PHYS Corpus Validation
- 22. References

1. Overview

M2 is the primary model format used by World of Warcraft for all 3D models rendered in-engine: player characters, creatures, spell visuals, doodads, weapons, and UI elements. It is used for everything except terrain (ADT) and world map objects (WMO).

An M2 model is not a single file. It is a **bundle** of related files, each carrying a different aspect of the model data:

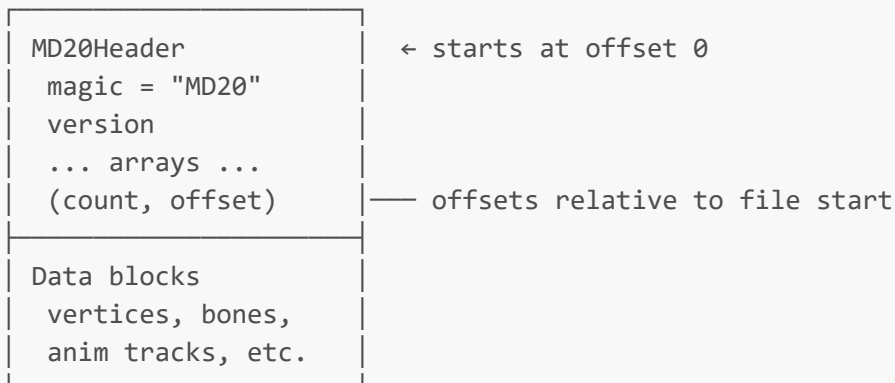
File	Purpose
<code>.m2</code>	Base model: header, geometry, bones, textures, materials, animations (or refs), cameras, lights, emitters, collision
<code>.skin</code>	LOD views: index buffers, submesh definitions, draw batches
<code>.anim</code>	Externalized animation keyframe data for low-priority sequences
<code>.bone</code>	Per-customization bone override matrices
<code>.skel</code>	Chunked skeleton: bones, attachments, sequences delegated from a parent skeleton hierarchy
<code>.phys</code>	Ragdoll physics: rigid bodies, shapes, joints

WhiteoutLib parses and writes all of the above. Physics reaches a model three ways — a `<stem>.phys` sibling by path, a `PFID` file id, or an inline `PFDC` chunk — and all three land in `Model::physics` as a parsed `PhysicsData`.

2. Format Variants

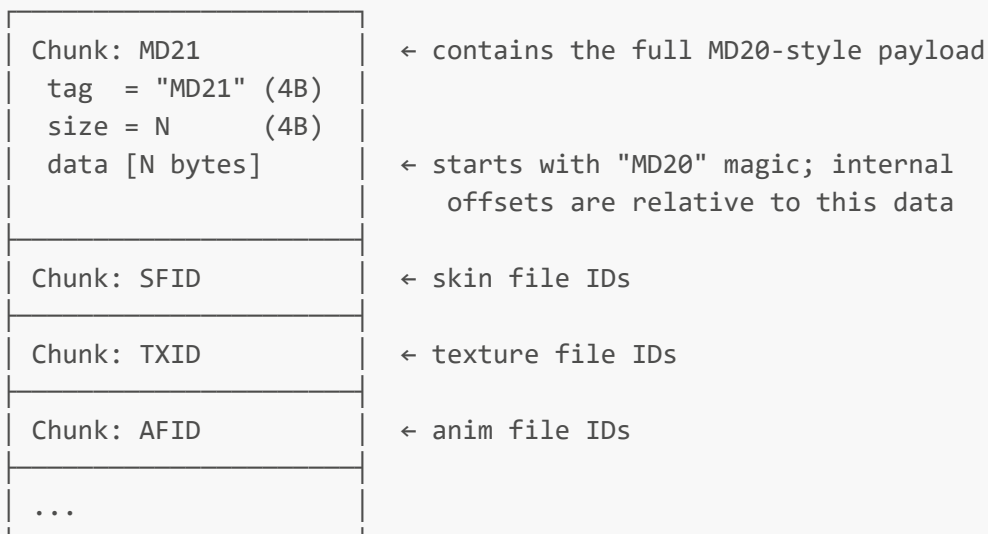
2.1 Classic (MD20)

Pre-Legion files begin directly with the four-byte magic `MD20` (`0x4D443230`). The entire file is a flat binary blob: a fixed-layout header followed by variable-length data blocks addressed through offset/count pairs.



2.2 Chunked (MD21)

Starting with Legion (7.0.1), the `.m2` file may use a chunked container. If the first four bytes are **not** `MD20`, the file is chunked. Chunks follow a uniform framing:



Important: Unlike every other chunked format in WoW, M2 chunk tags are **NOT byte-reversed** in the file. `AFID` in the file is `0x41464944`, reading left-to-right as ASCII.

Chunks may appear in any order. The MD21 chunk is typically first.

3. Version History

The **version** field in the MD20 header encodes a major.minor pair as $\text{major} * 256 + \text{minor}$. WhiteoutLib defines these constants:

Constant	Value	Major.Minor	Expansion(s)
<code>M2_VERSION_VANILLA</code>	256	1.0	Classic / Pre-Release
<code>M2_VERSION_BC</code>	260	1.4	The Burning Crusade
<code>M2_VERSION_WOTLK</code>	264	1.8	Wrath of the Lich King
<code>M2_VERSION_CATA</code>	265	1.9	Cataclysm
<code>M2_VERSION_MOP</code>	272	1.16	Mists of Pandaria
<code>M2_VERSION_WOD</code>	272	1.16	Warlords of Draenor
<code>M2_VERSION_LEGION</code>	272	1.16	Legion
<code>M2_VERSION_BFA</code>	273	1.17	Battle for Azeroth
<code>M2_VERSION_SL</code>	274	1.18	Shadowlands

Version ranges are inclusive. Models are version-tagged by the exporter; the version determines which structural fields and parsing branches are active.

Corpus note: The 9059-file corpus contains only version 272 (3629 files) and version 274 (5430 files). All files use the chunked MD21 format — no classic MD20 files were present.

Version-dependent code paths in WhiteoutLib:

Gate	Effect
<code>< BC (260)</code>	Bone rotation keys are <code>C4Quaternion</code> (4 × <code>f32</code>) instead of compressed <code>CompatQuaternion</code> ; particle <code>blendingType/emitterType</code> are each <code>u16</code> and there is no <code>particleColorIndex</code> ; skin sections end at <code>centerPosition</code> (no <code>sortCenterPosition/sortRadius</code>)

Gate	Effect
< WOTLK (264)	Tracks use the flat global-timeline layout (§5.6); sequences store <code>startTimestamp/endTimestamp</code> instead of <code>duration</code> ; header carries <code>playableAnimationLookup</code> (after the sequence lookup) and <code>textureFlipbooks</code> (after texture weights); skin profiles are inline in the <code>.m2</code> instead of external <code>.skin</code> files; no <code>.anim</code> siblings exist; ribbons end at the visibility track (no <code>priorityPlane/ribbonColorIndex/textureTransformIndex</code>); particles use the fixed three-point color block (§11.2) and a single <code>spin</code> float, with no <code>lifespanVariation/emissionRateVariation</code>
>= WOTLK (264)	Header stores <code>numSkinProfiles</code> as <code>u32</code> instead of inline <code>M2Array<SkinProfile></code> ; sequences whose <code>flags & 0x130 == 0</code> stream their keys from <code>.anim</code> siblings
< CATA (265)	Camera has a flat <code>f32 fieldOfView</code> before far/near clip; particles carry <code>u8 particleType + u8 headOrTail</code> where <code>multiTexScale[2]</code> later lives
>= CATA (265)	Camera has an <code>M2Track<f32> fieldOfViewTrack</code> after the roll track
< MOP (272)	Sequence blend time is a single <code>u32 blendTime</code> instead of the <code>u16 blendTimeIn/u16 blendTimeOut</code> pair (the split gave both sides the old value)
NewParticleRecord flag OR > 271	Particle emitter record is 492 bytes instead of 476: the extended <code>multiTexScrollMid[2][2] / multiTexScrollRange[2][2]</code> fields ($16 \times \text{fixed16_9} = 16$ bytes) are appended
<code>flag_use_texture_combiner_combos</code>	<code>textureCombinerCombos</code> array appended at header end

Pre-Legion models reference their sibling files by path rather than FileDataID: `<stem>%02d.skin`, `<stem>_LOD%02d.skin`, `<stem>%04d-%02d.anim` and — when the `LoadPhysicsData` global flag (0x100) is set — `<stem>.phys`. The naming was verified against the WoD 6.0.1 client's `CM2Shared::LoadSkinProfile / LoadLowPrioritySequence` format strings; matching is case-insensitive, as the client's own lookups are.

4. Primitive Types

WhiteoutLib defines these foundational types in `common_types.h`:

Scalar Types

Type	C++ Type	Size
<code>u8</code>	<code>uint8_t</code>	1
<code>u16</code>	<code>uint16_t</code>	2

Type	C++ Type	Size
u32	uint32_t	4
u64	uint64_t	8
i8	int8_t	1
i16	int16_t	2
i32	int32_t	4
f16	uint16_t (stored)	2
f32	float	4
f64	double	8

Normalized Integers

Type	Range	Mapping
snorm8 / snorm16 / snorm32	[-1.0, 1.0]	Signed integer → float via division by max absolute value
unorm8 / unorm16 / unorm32	[0.0, 1.0]	Unsigned integer → float via division by max value

The M2 format makes extensive use of **fixed16** (same as **i16** / **snorm16**) for alpha and weight values: **0x0000** = 0.0, **0x7FFF** = 1.0.

Vector / Matrix Types

Type	Layout	Size
Vector2f	f32 x, y	8
Vector3f	f32 x, y, z	12
Vector4f	f32 x, y, z, w	16
Quaternion	f32 x, y, z, w	16
PhysicsFrame	Vector3f axisX, axisY, axisZ, origin (column-major 3×4)	48
Matrix4x4	f32[16]	64

Compressed Quaternion (**CompatQuaternion**)

Post-Vanilla bone rotations use a compressed 8-byte quaternion:

```
struct CompatQuaternion {
    u16 x, y, z, w; // x,y,z are snorm16; w is unorm16
};
// Identity = (32767, 32767, 32767, 65535) → (0, 0, 0, 1)
```

XYZ components are signed normalized ($[-1, 1]$ mapped to $[0, 65534]$ with $32767 = 0$). The W component is unsigned normalized ($[0, 1]$ mapped to $[0, 65535]$).

Extent (Bounding Box + Sphere)

```
struct Extent {
    Vector3f minimum;    // 12 bytes
    Vector3f maximum;    // 12 bytes
    f32      sphereRadius; // 4 bytes
};                      // Total: 28 bytes
```

Used for both model bounding volumes and per-sequence animation bounds.

5. Core Patterns

5.1 M2Array

M2 data is organized through pointer-style arrays scattered throughout the file:

```
struct M2Array<T> {
    u32 count;    // number of elements
    u32 offset;   // byte offset to first element, relative to base
};              // Total: 8 bytes on disk
```

Offset base:

- **Classic MD20**: offsets are relative to the start of the file (offset 0)
- **Chunked MD21**: offsets are relative to the start of the MD21 chunk's **data** (after the 8-byte chunk header)

WhiteoutLib resolves these at parse time into `std::vector<T>`.

Runtime form (client memory, not the file): the 11.x client relocates each array **in place** rather than into a side table. It writes the resolved pointer over the `offset` field and stores the pointer's high 16 bits in the *upper half of the `count` field*, so an M2Array in a loaded model reads as:

```
struct M2ArrayResolved {
    u16 count;    // low half of the original u32 count
    u16 ptrHi;    // high 16 bits of the resolved 48-bit pointer
    u32 ptrLo;    // low 32 bits
};
// pointer = ptrLo | (u64(ptrHi) << 32)
```

The on-disk layout is unchanged — this only matters when comparing a memory dump against a file, and it is why the client reads element counts as `u16` throughout the chunk loader.

5.2 Animation Tracks

Animation data is the backbone of the M2 format. Nearly every animatable property uses this template:

```
struct M2TrackBase {
    u16 interpolationType;    // 0=None, 1=Linear, 2=Bezier, 3=Hermite
    u16 globalSequenceId;    // index into global_loops[], or 0xFFFF if none
    // >= Wrath: array-of-arrays
    M2Array<M2Array<u32>> timestamps; // [animation][keyframe] → ms
};

template<typename T>
struct M2Track : M2TrackBase {
    M2Array<M2Array<T>> values; // [animation][keyframe] → value
};
```

Each track has one sub-array per animation sequence. The outer index corresponds to the animation's position in the [sequences](#) array. If a sequence has no keyframes for this track, its sub-array is empty (count=0).

WhiteoutLib's in-memory representation:

```
struct AnimationTrackBase {
    u16 interpolationType;
    u16 globalSequenceId; // 0xFFFF = none
    std::vector<std::vector<u32>> timestamps;
};

template<typename T>
struct AnimationTrack : AnimationTrackBase {
    std::vector<std::vector<T>> values;
};
```

Pre-Wrath (< 264): Tracks use a single-timeline approach — one flat timestamp array and one flat value array, with an [interpolation_ranges](#) array mapping each animation to its begin/end indices. WhiteoutLib does not implement this path.

5.3 Interpolation

Type	Value	Behavior
None	0	Step function; value changes instantly at the timestamp
Linear	1	Linear interpolation (lerp for vectors/colors; nlerp for quaternions)
Bezier	2	Cubic Bézier spline using M2SplineKey (value + inTan + outTan)
Hermite	3	Cubic Hermite spline using M2SplineKey

For spline-based interpolation (types 2 and 3), keyframes use `SplineKey<T>`:

```
template<typename T>
struct SplineKey {
    u32 timestamp;
    T    value;
    T    inTangent;
    T    outTangent;
};
```

Note: There is historical ambiguity about whether type 2 is Bézier or Hermite. From WotLK onwards, type 2 = Bézier is confirmed by the community.

5.4 Global Sequences

Global sequences provide animation timing that is completely independent of the model's current animation. They are used for perpetually looping effects (weapon glows, ambient light pulses, etc.).

```
struct M2Loop {
    u32 timestamp; // maximum time in milliseconds
};
```

A track with `globalSequenceId != 0xFFFF` ignores per-animation timelines and instead loops within `[0, global_loops[globalSequenceId].timestamp]`. Global-sequence tracks always have exactly one sub-array (index 0).

5.5 ParticleAnimationTrack

Particle emitters animate properties over the lifetime of each emitted particle, keyed by the particle's **age** rather than by a model animation timeline. The track cannot vary between model animations. On disk it is two consecutive `M2Arrays` (no per-animation outer indirection, no interpolation/global-sequence header):

Offset	Type	Field	
0x00	u32	nTimestamps	
0x04	u32	ofsTimestamps	→ timestamps (u16, normalized to particle age)
0x08	u32	nValues	
0x0C	u32	ofsValues	→ T values

Unlike `M2Track`, the timestamps are `unorm16` values normalized to `[0, 1]` across the particle's lifetime (`0.0` = spawn, `1.0` = end of life) rather than millisecond keys, and there is no `interpolationType` / `globalSequenceId` prefix. WhiteoutLib parses these into `ParticleAnimationTrack<T>`:

```
template <typename T>
struct ParticleAnimationTrack {
    std::vector<unorm16> timestamps; // [0,1] over particle lifetime
```

```
std::vector<T>    values;
};
```

This type backs the per-lifetime color/alpha/scale/UV-scroll tracks on `ParticleEmitter` and the `alphaCutoff` track in the `EXP2` extension.

5.6 Legacy Tracks (versions < 264)

Before WotLK, a track was not split per sequence. All keys live in **one flat array on a global timeline**, and each sequence owns a `[startTimestamp, endTimestamp]` window of that timeline (stored in the sequence record, §7.2):

Offset	Type	Field	
0x00	u16	interpolationType	
0x02	u16	globalSequenceId	
0x04	M2Array<M2Range>	interpolationRanges	→ {firstKeyIndex, lastKeyIndex} per sequence,
			plus one trailing {0, 0} entry
0x0C	M2Array<u32>	timestamps	→ flat, global-timeline milliseconds
0x14	M2Array<T>	values	→ flat, same indexing as timestamps

That makes the legacy track header 28 bytes instead of 20 (24 vs 16 for the value-less event track).

WhiteoutLib converts on the fly, in both directions:

- **Reading:** each sequence's keys are the flat entries whose timestamp falls inside its window, re-based to the window start. The ranges array is redundant with the windows and is skipped. Global-sequence tracks (`globalSequenceId != 0xFFFF`) keep their whole span as sub-array 0 — their keys already run on the global loop's own clock.
- **Writing:** sequences are laid out onto a fresh global timeline with a 3333 ms gap between windows, keys are merged back with their window start added, and the ranges are regenerated. A sequence with zero or one keys in a track that has keys elsewhere is written as an equal-valued pair on its window boundaries — the layout old clients expect for constant sequences — so a legacy round-trip can grow such sub-tracks from one key to two playback-identical keys.

Bone rotation values in vanilla files (< 260) are float quaternions; they are compressed to `CompatQuaternion` with the client's exact mapping ($v < 0 ? v + 32768 : v - 32767$ per component, see §4) during conversion.

6. MD20 Header

6.1 Binary Layout

The header is a fixed-layout structure of count/offset pairs. Offsets point into the data block that follows the header. Record sizes are not stored — they are implicit from the structure definitions.

Offset	Type	Field
0x000	u32	magic ("MD20")
0x004	u32	version
0x008	M2Array<char>	name
0x010	u32	globalFlags
0x014	M2Array<M2Loop>	globalLoops
0x01C	M2Array<M2Sequence>	sequences
0x024	M2Array<u16>	sequenceIdxHashById
0x02C	M2Array<M2CompBone>	bones
0x034	M2Array<u16>	keyBoneLookup
0x03C	M2Array<M2Vertex>	vertices
0x044	u32	numSkinProfiles ($\geq$ WotLK)
0x048	M2Array<M2Color>	colors
0x050	M2Array<M2Texture>	textures
0x058	M2Array<M2TextureWeight>	textureWeights
0x060	M2Array<M2TextureTransform>	textureTransforms
0x068	M2Array<u16>	textureIndicesById
0x070	M2Array<M2Material>	materials
0x078	M2Array<u16>	boneCombos
0x080	M2Array<u16>	textureCombos
0x088	M2Array<u16>	textureCoordCombos
0x090	M2Array<u16>	textureWeightCombos
0x098	M2Array<u16>	textureTransformCombos
0x0A0	Extent	bounding
0x0BC	Extent	collision
0x0D8	M2Array<u16>	collisionTriangleIndices
0x0E0	M2Array<Vector3f>	collisionVertices
0x0E8	M2Array<Vector3f>	collisionFaceNormals
0x0F0	M2Array<M2Attachment>	attachments
0x0F8	M2Array<u16>	attachmentIndicesById
0x100	M2Array<M2Event>	events
0x108	M2Array<M2Light>	lights
0x110	M2Array<M2Camera>	cameras
0x118	M2Array<u16>	cameraIndicesById
0x120	M2Array<M2Ribbon>	ribbonEmitters
0x128	M2Array<M2Particle>	particleEmitters
--- conditional (if globalFlags & UseTextureCombinerCombos, $\geq$ BC) ---		
0x130	M2Array<u16>	textureCombinerCombos

WhiteoutLib stores this as **MD20Header**:

```
struct MD20Header {
    u32 magic;           // MD20
    u32 version;
    std::string modelName;
    GlobalFlags globalFlags;

    std::vector<GlobalSequence> globalLoops;
    std::vector<Sequence> sequences;
    std::vector<u16> sequenceIdxHashById;
```

```

std::vector<Bone> bones;
std::vector<u16> keyBoneIds;
std::vector<Vertex> vertices;
u32 numSkinProfiles;

std::vector<ColorAnimation> colors;
std::vector<Texture> textures;
std::vector<TextureWeight> textureWeights;
std::vector<TextureTransform> textureTransforms;
std::vector<u16> textureIndicesById;
std::vector<Material> materials;

std::vector<u16> boneCombos;
std::vector<u16> textureCombos;
std::vector<u16> textureCoordCombos;
std::vector<u16> textureWeightCombos;
std::vector<u16> textureTransformCombos;

Extent bounding;
Extent collision;
std::vector<u16> collisionTriangleIndices;
std::vector<Vector3f> collisionVertices;
std::vector<Vector3f> collisionFaceNormals;

std::vector<Attachment> attachments;
std::vector<u16> attachmentIndicesById;
std::vector<Event> events;
std::vector<Light> lights;
std::vector<Camera> cameras;
std::vector<u16> cameraIndicesById;
std::vector<RibbonEmitter> ribbonEmitters;
std::vector<ParticleEmitter> particleEmitters;
std::vector<u16> textureCombinerCombos; // if flag set
};

```

6.2 Global Flags

Stored as **u32** at offset **0x010** in the header. WhiteoutLib exposes these as **GlobalFlag**:

Flag	Bit	Description	Corpus Freq
TiltX	0x001	Model tilts along X axis	402 / 9059
TiltY	0x002	Model tilts along Y axis	118 / 9059
Unk_0x04	0x004	Unknown	60 / 9059

Flag	Bit	Description	Corpus Freq
UseTextureCombinerCombos	0x008	Appends <code>textureCombinerCombos</code> array at header end; multitexturing uses second material from this array	—
Unk_0x10	0x010	Unknown; extremely common in modern exports, purpose unclear	5664 / 9059
LoadPhysicsData	0x020	Model has associated physics data (<code>.phys</code> or <code>PFDC</code> chunk)	56 / 9059
Unk_0x80	0x080	Universal in modern (10.x/11.x/12.x) files; originally described as "DH tattoo glow" but set on every model in the corpus regardless of model type	9059 / 9059
CameraRelated	0x100	Camera-related flag; referenced by wowdev.wiki as "every model since Cata has this set", but corpus shows only 3 of 9059 files have it — likely only present on models with actual camera definitions	3 / 9059
NewParticleRecord	0x200	Extended 492-byte particle record (instead of 476 bytes)	7 / 9059
Unk_0x400	0x400	Unknown	—
TextureTransformsUsesBoneSequences	0x800	Texture transforms use the sequence played on a bone (looked up via <code>textureCoordCombos</code>) instead of the model's first bone	—
Unk_0x1000	0x1000	Unknown	2617 / 9059
ChunkedAnimFiles	0x2000	<code>.anim</code> files use chunked <code>AFM2/AFSA/AFSB</code> format	5867 / 9059
Unk_0x8000	0x8000	Unknown	512 / 9059
Unk_0x20000	0x20000	Unknown; may relate to ribbon emitter <code>textureTransformLookupIndex</code> usage (see ribbon emitter field documentation)	560 / 9059

Flag	Bit	Description	Corpus Freq
Unk_0x100000	0x100000	Unknown	61 / 9059
Unk_0x200000	0x200000	Unknown; very common in modern files	4274 / 9059
Unk_0x1000000	0x1000000	Unknown	21 / 9059
Unk_0x20000000	0x20000000	Unknown	53 / 9059
Unk_0x40000000	0x40000000	Unknown	14 / 9059

Bits not listed above (0x4000, 0x10000, 0x40000, 0x80000, etc.) were not observed in the 9059-file corpus but are preserved as raw flags by WhiteoutLib.

7. Skeleton and Animation

7.1 Global Sequence Loops

```
struct GlobalSequence {
    u32 timestamp; // maximum loop time in milliseconds
};
```

Referenced by tracks via `globalSequenceId`. If a track references a global sequence, its animation is independent of which model animation is playing.

7.2 Animation Sequences

```
struct Sequence {
    u16 id; // animation ID (references AnimationData.dbc)
    u16 variationIndex; // sub-animation index within the same ID
    u32 duration; // length in milliseconds (>= Wrath)
    f32 movespeed; // character movement speed during this animation
    u32 flags; // see Sequence Flags
    i16 frequency; // playback probability weight; all variations of an
    // ID sum to 0x7FFF (32767)

    u16 padding;
    u32 replayMin; // minimum repetition count (0 = no repeat)
    u32 replayMax; // maximum repetition count
    u16 blendTimeIn; // transition blend-in duration (ms)
    u16 blendTimeOut; // transition blend-out duration (ms)
    Extent bounding; // per-sequence bounding box
    i16 variationNext; // index of next variation of this animation ID, or -1
};
```

```

    u16 aliasNext;          // if this is an alias (flag 0x40), index of target
};                          // Total: 64 bytes (68 for versions < 264, see below)

```

The **blendTimeIn** / **blendTimeOut** split was introduced around WoD; older files had a single **u32 blendTime** in this slot (WhiteoutLib reads it into both fields, and writes **blendTimeIn** back as the **u32** for versions < 272). The client interpolates bone transforms between the ending state of one animation and the starting state of the next for the duration of the blend time.

Versions < 264 replace **duration** with a **u32 startTimestamp** / **u32 endTimestamp** pair (making the record 68 bytes): each sequence occupies that window on the shared global timeline that all legacy tracks are keyed against (§5.6). WhiteoutLib exposes **duration = end - start** and rebuilds the windows on write.

7.3 Sequence Flags

Flag	Value	Description	Corpus Freq
Unk_0x01	0x01	Unknown; seen commonly in sequences	19220 / 45310
Tilt In	0x02	Model starts upright, tilts over X/Y by end	21305 / 45310
Tilt Out	0x04	Model starts tilted, returns upright by end	21062 / 45310
Tilt Fixed	0x08	Model stays tilted for entire animation	20664 / 45310
Runtime loaded	0x10	Set at runtime for loaded low-priority sequences	21310 / 45310
Primary bone sequence	0x20	Animation data is embedded in the .m2 file (not externalized to .anim)	21583 / 45310
Is alias	0x40	This entry is an alias; follow aliasNext to find actual data	20528 / 45310
Blended	0x80	Lerp between end→start states on transition if values differ	22209 / 45310
Stored in model	0x100	Sequence data is stored within the model	19852 / 45310
Dual blend time	0x200	The 4-byte blend time field is two u16 (in + out)	19823 / 45310
Unk_0x400	0x400	Unknown	18908 / 45310
Unk_0x800	0x800	Seen in Legion 24500 models	19622 / 45310

Special flag patterns: The value `0x7FFF` (all bits 0–14 set) appears in 6347 sequences; these are typically alias sequences with all standard flags enabled. The value `0xFFFFFFFF` (-1 as `i32`) appears in 473 sequences and may serve as a sentinel. When the upper 16 bits of the flags `u32` are non-zero, the lower 16 bits are almost always `0xFFFF`, suggesting the upper half may carry auxiliary data (possibly a parent animation ID) rather than additional flags.

Corpus note: Across 9059 files (45310 total sequences), 10919 unique flag values were observed. 69% of sequences (31280) have the upper 16 bits as zero.

External animation loading rule: The client loads `.anim` files when `(flags & 0x130) == 0` — i.e., the animation is neither primary-bone-embedded (`0x20`), nor stored in model (`0x100`), nor has `0x10` set.

7.4 Sequence Index Hash (Animation Lookup)

```
u16 sequenceIdxHashById[]; // length = hash table size
```

A hash table mapping animation IDs to indices in the `sequences` array. The hash function is `anim_id % num_buckets` with quadratic probing (`stride2` increments). Value `-1` (`0xFFFF`) means "no entry". The referenced sequence is the first in its `aliasNext` chain for the given animation ID.

```
M2Sequence* find_entry(u32 anim_id) {
    size_t i = anim_id % sequenceIdxHashById.size();
    for (size_t stride = 1; ; ++stride) {
        if (sequenceIdxHashById[i] == 0xFFFF)
            return nullptr;
        if (sequences[sequenceIdxHashById[i]].id == anim_id)
            return &sequences[sequenceIdxHashById[i]];
        i = (i + stride * stride) % sequenceIdxHashById.size();
    }
}
```

7.5 Bones

```
struct Bone {
    i32 keyBoneId;        // index in key_bone_lookup, or -1 if not a key bone
    u32 flags;            // BoneFlag bits
    i16 parentBoneId;     // parent index, or -1 for root
    u16 submeshId;        // mesh part ID
    u32 boneNameCRC;      // CRC of bone name (debug only)

    AnimationTrack<Vector3f> translation;
    AnimationTrack<CompatQuaternion> rotation; // compressed; Vanilla used
                                                // full C4Quaternion

    AnimationTrack<Vector3f> scale;
    Vector3f pivot;       // pivot point in model space
};
```

The bone hierarchy defines the skeleton. Each bone's final transform is computed as $\text{parent_transform} * \text{translate}(\text{pivot}) * T * R * S * \text{translate}(-\text{pivot})$, where T/R/S come from the animation tracks.

Coordinate system: WoW uses a Z-up coordinate system. To convert to Y-up: $(x, y, z) \rightarrow (x, -z, y)$.

7.6 Bone Flags

Flag	Value	Description
IgnoreParentTranslate	0x001	Don't inherit parent's translation
IgnoreParentScale	0x002	Don't inherit parent's scale
IgnoreParentRotation	0x004	Don't inherit parent's rotation
SphericalBillboard	0x008	Bone always faces the camera (all axes)
CylindricalBillboardX	0x010	Billboard locked to X axis
CylindricalBillboardY	0x020	Billboard locked to Y axis
CylindricalBillboardZ	0x040	Billboard locked to Z axis
Transformed	0x200	Bone has animation transforms
Kinematic	0x400	Physics system can influence this bone (MoP+)
HelmetAnimScaled	0x1000	Helmet animation scaling via HelmetAnimScaling.dbc

Spherical and cylindrical billboard bits are mutually exclusive. Billboards are used for light halos, cannonball hemispheres, and similar view-facing geometry.

7.7 Key-Bone Lookup

```
u16 keyBoneLookup[]; // -1 (0xFFFF) if not present
```

Maps well-known bone roles to bone indices. The lookup index corresponds to a predefined bone role:

Index	Bone Role	Index	Bone Role
0	ArmL	1	ArmR
2	ShoulderL	3	ShoulderR
4	SpineLow	5	Waist
6	Head	7	Jaw
8–12	IndexFingerR through ThumbR	13–17	IndexFingerL through ThumbL
26	Root	27–34	Wheel1–8

Most character/creature models have 27–35 entries.

8. Geometry and Rendering

8.1 Vertices

```
struct Vertex {
    Vector3f      position;          // 12 bytes
    u8            boneWeights[4];    // 4 bytes  (sum should be 255)
    u8            boneIndices[4];    // 4 bytes  (into bones[])
    Vector3f      normal;            // 12 bytes
    Vector2f      texCoords[2];      // 16 bytes  (two UV sets)
};                                  // Total: 48 bytes
```

Vertices are stored globally in the `.m2` file. Skin profiles reference subsets of this vertex array. The two UV sets support multi-texturing; which set is used depends on the vertex shader selected by the rendering pipeline (e.g., `Diffuse_T1_T2` sends both sets, `Diffuse_T1_T1` sends the same UV for both textures).

8.2 Textures

```
struct Texture {
    u32          type;              // see Texture Types
    u32          flags;             // 0x1 = wrap X, 0x2 = wrap Y
    M2Array<char> filename;         // for type 0 only; includes null terminator
};
```

Type 0 textures have an in-file filename (BLP path). All other types are "hardcoded" — their actual texture is determined at runtime from database lookups (`CharSections.dbc`, `CreatureDisplayInfo.dbc`, `ItemDisplayInfo.dbc`, etc.). Non-type-0 textures still have a 1-byte filename pointing to a null terminator.

In chunked files ($\geq$ BfA), the `TXID` chunk provides `fileDataIDs` for textures, replacing filenames. The filename `M2Array` should be zeroed so the client falls through to the `TXID` lookup.

Implementation note: The filename buffer is stack-allocated at 0x108 bytes in the client, imposing an implicit 264-character path length limit.

8.3 Texture Types

Value	Name	Usage
0	NONE	Explicit filename / fileDataID
1	TEX_COMPONENT_SKIN	Body + clothes
2	TEX_COMPONENT_OBJECT_SKIN	Item, cape
3	TEX_COMPONENT_WEAPON_BLADE	Weapon blade / armor reflect
4	TEX_COMPONENT_WEAPON_HANDLE	Weapon handle
5	TEX_COMPONENT_ENVIRONMENT	Obsolete

Value	Name	Usage
6	TEX_COMPONENT_CHAR_HAIR	Character hair
7	TEX_COMPONENT_CHAR_FACIAL_HAIR	Obsolete
8	TEX_COMPONENT_SKIN_EXTRA	Skin extra detail
9	TEX_COMPONENT_UI_SKIN	Inventory art
10	TEX_COMPONENT_TAUREN_MANE	Character misc
11–13	TEX_COMPONENT_MONSTER_1–3	Creature/GO skins
14	TEX_COMPONENT_ITEM_ICON	Item icon
15–18	Guild background/emblem/border colors	> = Cata
19	Character Eyes	> = SL
20	Character Jewelry / Accessory	> = SL
21–24	Secondary skin/hair/armor + unknown	> = SL (not observed in 10.x/11.x corpus)
25–26	Unknown	> = DF (type 25: 4 files, type 26: 2 files in corpus)

8.4 Materials (Render Flags)

```
struct Material {
    u16 flags;           // see below
    u16 blendingMode;    // see M2/Rendering#M2BLEND
};
```

Material Flags:

Flag	Value	Meaning
Unlit	0x01	Not affected by lighting
Unfogged	0x02	Not affected by fog
TwoSided	0x04	Disable backface culling
DepthTest	0x08	Enable depth testing
DepthWrite	0x10	Enable depth writing
ShadowBatch1	0x40	Shadow batch related (WoD+)
ShadowBatch2	0x80	Shadow batch related (WoD+)
ShadowBatch3	0x200	Shadow batch related (WoD+)
NoAlphaComposite	0x800	Force opaque or fully transparent for custom elements (MoP+)

Blending modes (from M2/Rendering#M2BLEND):

Value	OpenGL Equivalent	Corpus Freq
0	<code>glDisable(GL_BLEND); glDisable(GL_ALPHA_TEST)</code>	2383
1	<code>glBlendFunc(GL_SRC_COLOR, GL_ONE)</code>	824
2	<code>glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)</code>	7354
3	<code>glDisable(GL_BLEND); glEnable(GL_ALPHA_TEST)</code>	—
4	<code>glBlendFunc(GL_SRC_ALPHA, GL_ONE)</code>	8743
5	Unknown (possibly <code>GL_DST_COLOR * ONE_MINUS_SRC_ALPHA</code>)	6
6	Unknown (possibly multiplicative or screen blend)	42
7	Unknown; very common in modern files — likely a Shadowlands+ blend mode	4133

8.5 Colors and Transparency

Color animations provide per-vertex color and alpha modulation:

```
struct ColorAnimation {
    AnimationTrack<Vector3f> color;    // RGB, [0,1] per component
    AnimationTrack<i16>      alpha;    // 0 = transparent, 0x7FFF = opaque
};
```

Texture weights specify global transparency in addition to color alpha:

```
struct TextureWeight {
    AnimationTrack<i16> weight;    // 0 = transparent, 0x7FFF = opaque
};
```

Both are referenced indirectly from skin batches via the combo/lookup tables. If a batch's `colorIndex` is -1, the submesh does not use this block.

8.6 Texture Transforms

```
struct TextureTransform {
    AnimationTrack<Vector3f>    translation;
    AnimationTrack<Quaternion> rotation;    // center = (0.5, 0.5)
    AnimationTrack<Vector3f>    scaling;
};
```

The rotation keys are **float quaternions** (`C4Quaternion`, 16 bytes per key) in every M2 version — unlike bone rotations, they were never compressed. This was verified against the client's `M2Init<M2TextureTransform>`, which routes the track through `M2Init<C4Quaternion>` (60-byte record: translation at +0, rotation at +20, scaling at +40).

Used for animated textures: flowing water, lava, pulsing runes, etc. The rotation is applied around the texture center $(0.5, 0.5)$:

1. Translate UV matrix to $(0.5, 0.5)$
2. Apply rotation
3. Translate UV matrix to $(-0.5, -0.5)$

8.7 Lookup / Combo Tables

The header contains several `u16[]` indirection tables that skin batches use to reference model-level resources:

Table	Alt Name	Purpose
<code>boneCombos</code>	<code>bone_lookup_table</code>	Bone indices for skinning; skin sections reference a slice
<code>textureCombos</code>	<code>texture_lookup_table</code>	Indices into the <code>textures[]</code> array
<code>textureCoordCombos</code>	<code>tex_unit_lookup_table</code>	UV mapping selection (-1=env, 0=UV0, 1=UV1)
<code>textureWeightCombos</code>	<code>transparency_lookup_table</code>	Indices into <code>textureWeights[]</code>
<code>textureTransformCombos</code>	<code>texture_transforms_lookup_table</code>	Indices into <code>textureTransforms[]</code>
<code>textureIndicesById</code>	<code>replacable_texture_lookup</code>	Replaceable texture lookup (type → texture index, or -1)
<code>textureCombinerCombos</code>	<code>Second_Texture_Material_Override_Combos</code>	Second-pass material override for multitexturing (flag-gated)

When `UseTextureCombinerCombos` is set, multitexturing reads the second texture's blend mode from `textureCombinerCombos[batch.shaderId]` instead of from `materials[batch.materialIndex + 1]`.

For models with multitexturing, lookup tables function in pairs: the first texture uses the mapping at the given index, the second texture uses the next index in sequence.

9. Bounding and Collision Volumes

The header `bounding` and `collision Extent` structures define the overall model bounds and a simplified collision volume.

Collision geometry is defined by three arrays:

Array	Content
<code>collisionVertices[]</code>	<code>Vector3f</code> positions

Array	Content
<code>collisionTriangleIndices[]</code>	<code>u16</code> indices (3 per triangle, count = number of indices)
<code>collisionFaceNormals[]</code>	<code>Vector3f</code> normals (1 per triangle, count = indices/3)

Characters and creatures typically have a simple box-like collision mesh.

10. Scene Objects

10.1 Attachments

```
struct Attachment {
    u32 id;                // attachment type (see table below)
    u16 boneId;            // anchor bone
    u16 unknown;
    Vector3f position;      // offset from bone; often same as bone pivot
    AnimationTrack<u8> animate; // enable/disable flag (bool); default = true
};
```

Attachments define mount points for weapons, effects, and UI elements on the model. The `id` determines the semantic role:

Attachment ID table (selected):

ID	Name	ID	Name	ID	Name
0	Shield / MountMain	11	Helm	26	SheathMainHand
1	HandRight	12	Back	27	SheathOffHand
2	HandLeft	17	Breath	28	SheathShield
5	ShoulderRight	18	PlayerName	29	PlayerNameMounted
6	ShoulderLeft	19	Base	34	Chest
7–10	KneeR/L, HipR/L	20	Head	39–46	VehicleSeat1–8 (Wrath+)
3	ElbowRight	21	SpellLeftHand	47	LeftFoot
4	ElbowLeft	22	SpellRightHand	48	RightFoot

For weapons, IDs 0–4 correspond to the 5 `ItemVisuals.dbc` columns for glow effects. The `PlayerName` (18) attachment is used by `CGUnit_C::GetNamePosition` for the name plate. `PlayerNameMounted` (29) takes priority when mounted.

The `attachmentIndicesById` lookup maps attachment type → index in `attachments[]`. Value -1 means not present.

10.2 Events

```

struct Event {
    u32 identifier;           // 4-char ID, usually '$' + 3 chars (e.g., "$DTH")
    u32 data;                 // context-dependent payload (often soundEntryId)
    u32 boneId;               // attachment bone
    Vector3f position;        // relative to bone
    AnimationTrackBase enabled; // timestamp-only track; each timestamp = "fire"
};

```

Events carry no value data — each timestamp in the **enabled** track is an implicit trigger. The **identifier** is a 4-byte ASCII code (stored as **u32**), and **data** carries a context-dependent payload (e.g., **soundEntryId** for sound events, **SpellEffectCameraShakes.dbc** ID for screen shakes).

Common events:

ID	Purpose	Triggered On
\$DTH	Death thud + loot effect	Death, Drown, Knockdown
\$FSD	Footstep sound	Walk, Run
\$CST	Release missiles	Attack/Spell casts
\$CSL / \$CSR	Release missiles left/right	Attack/Spell casts
\$BWR	Bow release	AttackRifle, AttackBow
\$BWP	Bow pull	LoadRifle, LoadBow
\$SHL / \$SHR	Sheathe weapon left/right	Sheath, HipSheath
\$BTH	Breath effect	Idle (underwater, snow)
\$CSS	Weapon swoosh sound	Attack animations
\$HIT	Wound anim kit	Attack animations
\$FL0-\$FL3	Footstep left	Walk/Run
\$FR0-\$FR3	Footstep right	Walk/Run
\$CSD	Emote sound (data=soundEntryId)	Emote animations
\$DSE	Destroy emitter	Map objects
\$DSO	Doodad sound one-shot (data=soundEntryId)	Game objects
\$SHK	Camera shake (data=cameraShakeId)	Game objects

10.3 Lights

```

struct Light {
    u16 type;           // 0 = Directional, 1 = Point
    i16 boneId;         // -1 if not bone-attached
    Vector3f position; // relative to bone
};

```

```

    AnimationTrack<Vector3f> ambientColor;
    AnimationTrack<f32>      ambientIntensity; // default 1.0
    AnimationTrack<Vector3f> diffuseColor;
    AnimationTrack<f32>      diffuseIntensity; // default 1.0
    AnimationTrack<f32>      attenuationStart;
    AnimationTrack<f32>      attenuationEnd;
    AnimationTrack<u8>       visibility;        // 0/1 toggle
};

```

Type	Value	Usage
Directional	0	Login screens only (not used in world)
Point	1	Wands, doodads, creature effects

In TBC (and possibly other versions), the client hardcodes constant attenuation to 0, linear attenuation to 0.7, and quadratic attenuation to 0.03 for login screen lights.

10.4 Cameras

```

struct Camera {
    u32 type; // 0=portrait, 1=characterinfo, -1=flyby
    f32 fieldOfView; // diagonal FOV in radians (< Cata only)
    f32 farClip;
    f32 nearClip;

    AnimationTrack<CameraSpline> positions;
    Vector3f positionBase;
    AnimationTrack<CameraSpline> targetPositions;
    Vector3f targetPositionBase;
    AnimationTrack<f32> roll; // 0 to 2π
    AnimationTrack<f32> fieldOfViewTrack; // >= Cata only
};

struct CameraSpline {
    Vector3f value;
    Vector3f inTangent;
    Vector3f outTangent;
};

```

Cameras are present in most models for the character portrait and character info tab. Flyby cameras (type -1) are used in geometry-less camera-path models and login screen backdrops.

FOV conversion: The M2 stores diagonal FOV (d_{fov}). Convert to vertical FOV (v_{fov}):

$$v_{fov} = \frac{d_{fov}}{\sqrt{1 + aspect^2}}$$

The `cameraIndicesById` lookup maps type → camera index. `cameraIndicesById[1]` is the character info tab camera. A lookup value of -1 means "not referenced."

11. Effects

11.1 Ribbon Emitters

```
struct RibbonEmitter {
    u32 ribbonId;                // usually -1
    u32 boneId;
    Vector3f position;           // relative to bone

    std::vector<u16> textureIndices; // into textures[]
    std::vector<u16> materialIndices; // into materials[]

    AnimationTrack<Vector3f> colorTrack;    // RGB multiplier
    AnimationTrack<i16>      alphaTrack;    // 0=transparent, 0x7FFF=opaque
    AnimationTrack<f32>      heightAboveTrack;
    AnimationTrack<f32>      heightBelowTrack;

    f32 edgesPerSecond;           // smoothness; quads generated per second
    f32 edgeLifetime;             // seconds quads persist
    f32 gravity;                  // arcsin(val) = emission angle in degrees

    u16 textureRows;              // texture atlas rows
    u16 textureCols;              // texture atlas columns

    AnimationTrack<u16> texSlotTrack;
    AnimationTrack<u8>  visibilityTrack;

    i16 priorityPlane;            // >= Wrath
    i8  ribbonColorIndex;
    i8  textureTransformLookupIndex; // into textureTransformCombos;
                                         // only used if globalFlags & 0x20000
};
```

Ribbons generate trailing quads along the path of the attached bone. `heightAbove/heightBelow` define the ribbon width (they should differ). `edgesPerSecond` controls smoothness (low value = more jagged), and `edgeLifetime` controls trail length.

Examples: wisps in Blackfathom Deeps, Al'ar the Phoenix, Caverns of Time entrance light-trails.

11.2 Particle Emitters

Particle emitters are the most complex M2 structure. The base record is 476 bytes, or 492 bytes for the extended record (see below). WhiteoutLib parses it into `ParticleEmitter`, listed here in exact on-disk field order:

```

struct ParticleEmitter {
    u32          particleId = UINT32_MAX;    // usually -1
    ParticleFlag flags;                     // 32-bit flag set (see Particle
Flags)
    Vector3f     position;                   // emitter position relative to bone
    u16          boneId;
    union {                                  // Cata+ packs 3 texture indices
        u16 textureId;
        struct { u16 textureId1:5, textureId2:5, textureId3:5, padding:1; };
    };

    std::string particleModelFilename;      // model-geometry particles (if non-
empty)
    std::string childEmittersModelFilename; // trail / child-emitter model per
particle

    ParticleBlending    blendingType;        // u8 (0-4); u16 in versions < 260
    ParticleEmitterType emitterType;        // u8 (1=Plane, 2=Sphere, 3=Spline,
4=Bone); u16 in versions < 260
    u16          particleColorIndex;        // 11/12/13 → ParticleColor.dbc row;
absent < 260

    u8          particleType, headOrTail;   // versions < 265 only, in the
multiTexScale slot
    fixed8_5    multiTexScale[2];          // per-layer texture scale (1 byte
each); >= 265
    i16         textureTilerotation;        // -1, 0, or 1 – also serves as
priorityPlane
    u16         rows;                       // texture atlas rows
    u16         columns;                    // texture atlas columns

    // Emitter-lifetime animated tracks (full M2Track, keyed in ms):
    AnimationTrack<f32> emissionSpeed;
    AnimationTrack<f32> speedVariation;     // [0, 1]
    AnimationTrack<f32> verticalRange;      // [0,  $\pi$ ]
    AnimationTrack<f32> horizontalRange;    // [0,  $2\pi$ ]
    AnimationTrack<f32> gravity;
    AnimationTrack<f32> lifespan;
    f32         lifespanVariation;         // life += lifespanVariation *
rand(-1,1)
    AnimationTrack<f32> emissionRate;
    f32         emissionRateVariation;
    AnimationTrack<f32> emissionAreaWidth;  // plane: Y width, sphere: max radius
    AnimationTrack<f32> emissionAreaLength; // plane: X width, sphere: min radius
    AnimationTrack<f32> zSource;

    // Per-particle-lifetime tracks (ParticleAnimationTrack; timestamps unorm16
[0,1]):
    ParticleAnimationTrack<Vector3f> colorTrack;    // RGB over lifetime
    ParticleAnimationTrack<unorm16> alphaTrack;     // opacity over lifetime
    ParticleAnimationTrack<Vector2f> scaleTrack;    // size over lifetime
    Vector2f          scaleVary;                   // {x, y} random scale

```

```

variation
    ParticleAnimationTrack<unorm16> headUVScroll; // head flipbook cell over
lifetime
    ParticleAnimationTrack<unorm16> tailUVScroll; // tail flipbook cell over
lifetime

    f32      tailLength;
    f32      twinkleSpeed;           // blink speed (twinkleFPS)
    f32      twinklePercent;        // fraction of time visible (1.0 =
always)
    Vector2f twinkleScale;           // {min, max} scale variation
    f32      inheritVelocityScale;   // scales velocity inherited from
parent
    f32      drag;                   // speed *= exp(-drag * t)

    f32      baseSpin, baseSpinVariation; // initial 2D billboard spin angle
    f32      spinSpeed, spinSpeedVariation; // 2D billboard rotation per second

    M2Box    tumble;                 // model-particle angular velocity
{min, max}
    Vector3f  windVector;            // static wind (ignored if
DynamicWind set)
    f32      windTime;

    f32      followSpeed1, followScale1;
    f32      followSpeed2, followScale2;

    std::vector<Vector3f> splinePoints; // path for Spline emitter type
    AnimationTrack<u8>    enabledIn;    // visibility toggle

    // Extended record only – version > 271 OR (globalFlags & NewParticleRecord):
    fixed16_9 multiTexScrollMid[2][2]; // per-layer UV scroll center
    fixed16_9 multiTexScrollRange[2][2]; // per-layer UV scroll range

    std::optional<ParticleEmitterExtension> extension; // from EXP2 / EXPT chunk
};

```

Versions < 264 have no **lifespanVariation** / **emissionRateVariation**, a single **f32 spin** in place of the four spin floats, and replace the five per-lifetime tracks + **scaleVary** with a fixed three-point block (52 bytes):

```

f32      midPoint;           // mid key's position in the particle lifetime [0, 1]
u8[4][3] colorValues;       // BGRA at begin / mid / end
f32[4]   scaleValues;       // uniform scale at begin / mid / end (+1 unused)
u16[2]   headCellBegin;    u16 pad;
u16[2]   headCellEnd;      u16 pad;
i16[4]   tiles;            // tail flipbook cells

```

WhiteoutLib converts this to/from three-key **ParticleAnimationTracks** at timestamps **{0, midPoint, 1}** (colors as 0–255 floats, alpha widened to fixed-15, scalar scale duplicated into both **Vector2f** lanes). Writing samples the tracks back at begin/mid/end.

Field-order notes (vs. older docs and wowdev.wiki):

- WhiteoutLib reads **emissionAreaWidth** before **emissionAreaLength** (the reverse of some references). The parser and writer agree, so round-trips are byte-exact.
- The old **burstMultiplier** slot is parsed as **inheritVelocityScale** (the value scaling velocity inherited from a parent emitter).
- The per-lifetime color/alpha/scale/UV tracks are **ParticleAnimationTracks** keyed by particle age, **not** full **M2Tracks** — see §5.5.
- **multiTexScale** ($2 \times \text{fixed8_5}$) and the extended **multiTexScrollMid** / **multiTexScrollRange** (**fixed16_9**) drive multi-texture (Cata+) UV animation.

Emitter types (**ParticleEmitterType**):

Value	Name	Description
1	Plane	Emit from a rectangular area (uses emissionAreaWidth/Length)
2	Sphere	Emit from a sphere shell (Width =max radius, Length =min radius)
3	Spline	Emit along splinePoints
4	Bone	Emit from bone position (BoneGeneratorBone flag selects bone vs joint)

Blending modes (**ParticleBlending**):

Value	Name
0	Opaque (no blend, no alpha test)
1	AlphaBlend (SRC_ALPHA + ONE_MINUS_SRC_ALPHA)
2	Additive (SRC_ALPHA + ONE)
3	AlphaTest (no blend)
4	AdditiveAlphaTest

Compressed Particle Gravity (flag **0x800000**): Gravity keyframe values are stored as 4 bytes (**i8 x**, **i8 y**, **i16 z**) instead of a float. At load time:

```
CompressedParticleGravity v = *(CompressedParticleGravity*)pValue;
Vector3f dir = Vector3f(v.x, v.y, 0) * (1.0f / 128.0f);
float z = sqrtf(1.0f - dot(dir, dir));
float mag = v.z * 0.04238648f;
if (mag < 0) { z = -z; mag = -mag; }
dir.z = z;
dir *= mag;
```

Without the flag, gravity is a simple **float** applied as **(0, 0, -gravity)**.

11.3 Particle Flags

ParticleFlag is a 32-bit set. Names and meanings below match the WhiteoutLib enum, which was reconstructed from the client's **CParticleEmitter2** / **CParticleMat** logic; descriptions note that behavior where known.

Flag	Value	Description
Unlit	0x1	Disables lighting (sets the material unlit bit); particles are lit by default
SortParticles	0x2	Depth-sorted rendering via priority queue
VelocityOrient	0x4	Billboard aligns along the velocity vector
Unfogged	0x8	Not affected by fog (sets the material unfogged bit)
WorldSpace	0x10	Particles operate in world space; skips the bone-matrix transform
InheritBoneScale	0x20	Scale particles by attached bone (sqrt of bone-matrix column length)
InheritVelocity	0x40	Child emitter inherits parent velocity (scaled by inheritVelocityScale)
ImplosionFilter	0x80	Kill particles moving away from the center
HemisphereUpDirection	0x100	Force Z-up velocity direction in sphere emitters
NegateSpinRandom	0x200	Negate spin angle for particles whose random bit & 1
ClampTailToAge	0x400	Clamp tail length to min(tailLength, age)
InheritPosition	0x800	Child inherits parent position; random emission spacing
XYQuad	0x1000	Use the quad-to-view matrix instead of a screen-aligned billboard
ProjectParticle	0x2000	Snap particle to terrain via the project callback
FollowPosition	0x4000	Add emitter delta-position to particle when 2*dt < age
Squirt	0x8000	Burst emission only when emissionRate is animated
ChooseRandomTexture	0x10000	Random flipbook frame
HeadStyle	0x20000	Head particle style bits in SetParticleStyle
TailStyle	0x40000	Tail particle style bits in SetParticleStyle
UnscaledSizeVariation	0x80000	Independent X/Y scale variation (two random floats)
Refraction	0x100000	Refraction particle (type 3): drawn with the Particle_Refraction shader, forced unlit, not rendered while the camera is submerged; MultiTexture takes precedence when both are set
RandFlipbookStart	0x200000	Random starting flipbook frame

Flag	Value	Description
Unk_0x400000	0x400000	Unknown — present in data but never read by the client
CompressedGravity	0x800000	Gravity keyframes are compressed direction vectors (int8 <i>x</i> , <i>y</i> + int16 <i>z</i>) instead of z-axis floats
BoneGeneratorBone	0x1000000	Select bone (1) vs joint (0) generator; only consulted when emitterType is Bone (4), meaningless otherwise
NoGlobalViewScale	0x2000000	Skip the global-view-scale multiply on emission rate
LodIgnoreDistance	0x4000000	Skip distance-based LOD emission-rate scaling
OffsetHeadBySpin	0x8000000	Offset head particle position along the spin rotation axis
MultiTexture	0x10000000	Route through the multi-texture particle path (Cata+)
MultitexUseModx4	0x20000000	CParticleMat bit 3; Modx4 instead of Modx2 (requires MultiTexture)
MultitexUse3Colors	0x40000000	CParticleMat bit 4; 3 colors instead of 2 (requires MultiTexture)
DynamicWind	0x80000000	Enable dynamic wind callback; clear = static wind from windVector

12. SKIN File (.skin)

Skin files define Level-of-Detail (LOD) views onto the model's vertex data. Starting from WotLK, they are stored externally; earlier versions embedded them in the .m2 file. Each .skin file contains a single **SkinProfile**.

12.1 Skin Profile

The header is 64 bytes for Cata+ models (48 bytes pre-Cata):

Offset	Size	Type	Field
-----	----	----	-----
0	4	u32	magic ("SKIN" = 0x534B494E)
4	8	M2Array<u16>	vertices
12	8	M2Array<u16>	indices
20	8	M2Array<u8[4]>	bones
28	8	M2Array<SkinSection>	submeshes
36	8	M2Array<Batch>	batches
44	4	u32	lodVertexBase
48	8	M2Array<ShadowBatch>	shadowBatches (>= Cata)
56	4	u32	reserved (always 0) (>= Cata)
60	4	u32	reserved (always 0) (>= Cata)

```
struct SkinProfile {
    u32 magic; // "SKIN" (0x534B494E)
```

```

std::vector<u16> vertices;           // indices into M2 vertex array
std::vector<u16> indices;           // triangle indices into vertices[]
std::vector<std::array<u8,4>> bones; // per-vertex bone remapping
std::vector<SkinSection> submeshes;
std::vector<Batch> batches;

u32 lodVertexBase;    // cumulative vertex base for LOD skins (always 0 for
skin00)

std::vector<ShadowBatch> shadowBatches; // >= Cata
// 8 bytes reserved (always 0)
};

```

Offsets within a **.skin** file are relative to the start of the file. The **vertices** array is a level of indirection: **skinProfile.indices[i]** is an index into **skinProfile.vertices[]**, and **skinProfile.vertices[j]** is an index into the M2's global **vertices[]**.

lodVertexBase — HD Model Vertex Splitting

All skin files use **u16** vertex indices and **u16** triangle indices — there is no u32 variant. Models with more than 65,535 total M2 vertices (e.g., HD character models with 200k–360k vertices) split their vertex data across multiple LOD skins, with each skin referencing a window of ≤65,535 vertices. The **lodVertexBase** field provides the offset into the M2's global vertex array for each skin:

- **Base skin (skin00):** **lodVertexBase** = 0. Vertex indices reference M2 global vertices [0, vertexCount).
- **LOD skin N:** **lodVertexBase** = sum of **vertexCount** from skins 0 through N–1. Vertex indices reference M2 global vertices [**lodVertexBase**, **lodVertexBase** + **vertexCount**).

Example — **drachthyrfemale** (359,270 total M2 vertices):

Skin File	lodVertexBase	Skin Verts	Global Range
skin00	0	61,934	0 – 61,933
lod01	61,934	61,934	61,934 – 123,867
lod02	123,868	55,407	123,868 – 179,274
lod03	179,275	50,391	179,275 – 229,665
lod04	229,666	46,263	229,666 – 275,928
lod05	275,929	42,342	275,929 – 318,270
lod06	318,271	40,999	318,271 – 359,269

Corpus (14,443 skin files): 6,779 non-LOD skins all have **lodVertexBase** = 0; all 4,867 LOD skins have **lodVertexBase** > 0 matching the cumulative vertex chain.

12.2 Skin Sections (Submeshes)

```

struct SkinSection {
    u16 skinSectionId;        // submesh group identifier
    u16 level;                // LOD level (0 = highest detail)
    u16 vertexStart;          // start in skin's vertices[]
    u16 vertexCount;
    u16 indexStart;           // start in skin's indices[]
    u16 indexCount;
    u16 boneCount;            // bones used by this submesh
    u16 boneComboIndex;       // start in header's boneCombos[]
    u16 boneInfluences;       // max bones per vertex (1-4)
    u16 centerBoneIndex;      // bone for distance calculation
    Vector3f centerPosition;
    Vector3f sortCenterPosition; // >= BC (260); vanilla sections end at
                                // centerPosition
    f32 sortRadius;           // >= BC (260)
};

```

The `skinSectionId` groups submeshes for geoset-based visibility toggling (e.g., hairstyles use different geoset IDs, and only one is active at a time based on equipped items or character options).

12.3 Batches (Texture Units)

```

struct Batch {                // 24 bytes
    u8  flags;
    i8  priorityPlane;
    u16 shaderId;
    u16 skinSectionIndex;     // into submeshes[]
    u16 geosetIndex;
    i16 colorIndex;           // into header's colors[], or -1
    u16 materialIndex;        // into header's materials[]
    u16 materialLayer;
    u16 textureCount;
    u16 textureComboIndex;    // into header's textureCombos[]
    u16 textureCoordComboIndex; // into header's textureCoordCombos[]
    u16 textureWeightComboIndex;
    u16 textureTransformComboIndex;
};

```

Each batch represents a draw call. The `shaderId` selects which vertex/pixel shader combination to use and which UV sets to pass. When `UseTextureCombinerCombos` is set, `shaderId` also indexes into the `textureCombinerCombos` array for the second texture's blend mode.

Material, texture, and animation indices use the combo tables as indirection layers, enabling multiple skin sections to share resources efficiently.

Batch entries are always 24 bytes. The batch data array is followed by standard 16-byte alignment padding before the shadow batch data. For odd batch counts, $\text{batch_count} * 24 \equiv 8 \pmod{16}$, producing 8 bytes

of zero padding. This padding should not be confused with an extended 32-byte batch element size — there are no 32-byte batch entries in the corpus.

12.4 Shadow Batches

```
struct ShadowBatch {
    u8  flags;
    u8  flags2;          // 0x8 = use EDGF edge fade data
    u16 unknown0;
    u16 submeshId;
    u16 textureId;
    u16 colorId;
    u16 transparencyId;
};
```

Present from Cata onwards. Defines the shadow rendering pass for submeshes. `flags2 & 0x8` triggers edge-fade rendering using the `EDGF` chunk. Shadow batch data immediately follows the batch data region in the file.

Corpus (3,968 files with shadow batches, 66,245 total shadow batch entries):

flags value	Count	flags2 value	Count
0x10	56,698	0x02	42,358
0x90	7,216	0x03	14,511
0x80	1,406	0x00	5,778
0x00	421	0x01	3,415
0xC0	149	0x05	72
0x18	102	0x07	48

12.5 HD Model Identification

HD models can be identified via the M2 global flags. The strongest discriminator is **bit 20** (`0x00100000`):

Flag bit	HD models	Non-HD models	Notes
bit 20 (<code>0x00100000</code>)	72.0%	2.2%	Strongest HD indicator
bit 4 (<code>0x00000010</code>)	0.0%	76.4%	Never in HD
bit 12 (<code>0x00001000</code>)	0.0%	47.6%	Never in HD
bit 21 (<code>0x00200000</code>)	88.0%	82.0%	Not discriminating

No single bit is a perfect binary classifier. HD models typically carry flag `0x00302080` (bit 7 + bit 13 + bit 20 + bit 21), while non-HD models commonly carry `0x00202080` or `0x00202090`.

13. Chunked M2 (MD21 + Chunks)

13.1 Chunk Format

```
struct Chunk {
    char tag[4];    // ASCII, NOT byte-reversed (unlike other WoW formats)
    u32 size;       // payload size in bytes (excludes the 8-byte header)
    u8 data[size];
};
```

Chunks can appear in any order. WhiteoutLib's parser handles them order-independently via a dispatch loop. The writer emits them in a fixed deterministic order.

13.2 Chunk Catalogue

WhiteoutLib implements 30 chunk types. The following catalogue includes corpus statistics from 9059 M2 files:

Chunk	Since	Description	Corpus Files	Size Range
MD21	Legion	Core model data (MD20 payload)	9059	512–38 MB
PFID	Legion	Physics .phys fileDataId	—	4
SFID	Legion	Skin + LOD skin fileDataIds	9059	4–32
AFID	Legion	Animation .anim fileDataIds	109	8–592
BFID	Legion	Bone .bone fileDataIds	26	20–120
TXAC	Legion	Texture combiner parameters	4631	2–474
EXPT	Legion	Extended particle data v1	55	12–120
EXP2	Legion 7.3	Extended particle data v2	3138	48–14 KB
PABC	Legion 7.3	Parent sequence blacklist	1	32
PADC	Legion 7.3	Parent texture weights	9	21–49 KB
PSBC	Legion 7.3	Parent sequence bounds	25	2.5–11 KB
PEDC	Legion 7.3	Parent event data	25	29–108 KB
SKID	Legion 7.3	Skeleton .skel fileDataId	61	4
TXID	BfA	Texture fileDataIds (replaces filenames)	9019	4–220
LDV1	BfA	LOD configuration (16 bytes)	3630	16
RPID	BfA 8.1	Recursive particle model fileDataIds	4	8–24
GPID	BfA 8.1	Geometry particle model fileDataIds	12	8–84
WV1	BfA 8.2	Waterfall/PBR marker v1	—	0

Chunk	Since	Description	Corpus Files	Size Range
WV2	BfA 8.2	Waterfall/PBR marker v2	—	0
PGD1	Classic 1.13	Particle geoset data (M2Array-style)	2077	32–480
WV3	SL	Waterfall/PBR data v3	—	~96
PFDC	SL	Inline physics data	19	352–5584
EDGF	SL	Edge fade parameters (24 B/entry)	685	32–624
NERF	SL	Distance-based alpha (16 bytes)	1	16
DETL	SL	Detail light parameters (12 B/entry)	278	16–64
DBOC	SL	Distance-based opacity control (16 B/entry)	1022	16–512
AFRA	DF	Animation frame rate	—	—
PCOL	11.1.7	Player housing collision mesh	14	384–11 KB
DPIV	11.1.7	Pivot displacement data (32 B/entry)	15	32–64
TEXL	12.0.0	Textured light parameters (16 B/entry)	3	16

Corpus note: No unknown chunk tags were found in the 9059-file corpus. All chunks belong to the 30 documented types above. WV1, WV2, WV3, AFRA, and PFID were not observed (the corpus may not include waterfall models or physics-heavy models).

13.3 File ID Reference Chunks

These chunks replace filename-based file discovery with integer file data IDs (CASC storage):

PFID — Physics:

```
struct PFIDChunk {
    u32 physFileDataId;
};
```

SFID — Skins:

```
struct SFIDChunk {
    std::vector<u32> skinFileDataIds;    // count = numSkinProfiles
    std::vector<u32> lodSkinFileDataIds; // remaining entries
};
```

Some models (e.g., [NightborneFemaleCitizen.m2](#)) have fewer bytes in this chunk than expected — 4 skin files + 2 LOD files but only 20 bytes (4 skins + 1 LOD).

AFID — Animations:

```

struct AFIDEntry {
    u16 animId;
    u16 subAnimId;
    u32 fileDataId;    // 0 = none
};
struct AFIDChunk {
    std::vector<AFIDEntry> entries;
};

```

BFID — Bones:

```

struct BFIDChunk {
    std::vector<u32> boneFileDataIds;
};

```

SKID — Skeleton:

```

struct SKIDChunk {
    u32 skeletonFileDataId;
};

```

TXID — Textures (replaces in-file filenames):

```

struct TXIDEntry {
    u32 fileDataId;
};
struct TXIDChunk {
    std::vector<TXIDEntry> entries;
};

```

RPID / GPID — Replace the particle emitter's inline model filenames with fileDataIds (RPID → *childEmittersModelFilename*, GPID → *particleModelFilename*):

```

struct M2RPIDChunk { std::vector<M2RPIDEntry> entries; }; // fileDataId per
emitter
struct GPIDChunk    { std::vector<GPIDEntry>    entries; };

```

13.4 Particle Extension Chunks

TXAC — Texture/particle combiner hints:

```
struct TXACChunk {
    std::vector<std::array<u8, 2>> entries;
    // count = materials.count + particles.count
};
```

Likely used in `CM2SceneRender::SetupTextureTransforms`. Internally influences `CParticleEmitter2` vertex buffer format selection.

EXPT — Extended particle v1 (possibly superseded by EXP2):

```
struct EXPTEntry {
    f32 zSource;
    f32 colorMult;
    f32 alphaMult;
};
```

If EXP2 doesn't exist, the client reconstructs it from EXPT data.

EXP2 — Extended particle v2 (one entry per particle emitter):

```
struct ParticleEmitterExtension { // WhiteoutLib in-memory + on-disk layout
    f32 zSource;
    f32 colorMult;                // multiplies the particle's diffuse color
    f32 alphaMult;                // multiplies the particle's opacity
    ParticleAnimationTrack<unorm16> alphaCutoff; // per-lifetime alpha test
};
```

The `alphaCutoff` track is a `ParticleAnimationTrack` indexed by the particle's current lifetime position (timestamps are `unorm16` in `[0, 1]`).

In-memory mapping: WhiteoutLib surfaces both extension chunks through the optional `ParticleEmitter::extension` field, aligned by index with `particleEmitters[]`:

- When **EXP2** is present, each `emitterExtensions[i]` is moved directly into `particleEmitters[i].extension`.
- When only **EXPT** is present, the client (and WhiteoutLib) reconstruct the extension from EXPT: `zSource/colorMult/alphaMult` are copied and `alphaCutoff` is left empty.

PGD1 — Particle geoset assignment:

```
struct PGD1Chunk {
    // M2Array-style header:
    u32 count;                // number of geoset entries (= particle emitter count)
    u32 dataOffset;           // byte offset from chunk data start to geosets array
                                // (always 16 in observed files)
    u32 reserved[2];          // always 0
};
```

```

    // At byte `dataOffset`:
    u16 geosets[count]; // per-emitter geoset ID (0 = always visible)
    // Padding to 16-byte alignment
};
// Total size = 16 + ceil(count * 2 / 16) * 16
// Minimum observed size = 32 bytes (for 1-8 particles)

```

The chunk uses an inline `M2Array<u16>`-style layout (count + offset at bytes 0–7) with the geoset data at the declared offset. Each `u16` value assigns the particle emitter to a geoset group, obeying the same geoset visibility rules as `SkinSection.skinSectionId`. A value of `0` means the emitter is always visible. In the corpus (2077 files), most geoset entries are `0`; sparse non-zero values (e.g., 602, 206, 306) correspond to specific geoset IDs used for particle visibility gating.

13.5 Parent Data Chunks

Used when a model has a parent skeleton and needs to override or supplement parent animation data:

PABC — Parent sequence blacklist (called "BlacklistAnimData" internally):

```

struct PABCChunk {
    std::vector<u16> animationIds;
};

```

A flat array of animation IDs. If a target animation is found in this array, the parent's sequence lookup is bypassed. Otherwise, `parentSequencesLookups` are used.

PADC — Parent texture weights (replacement for `header.texture_weights`):

```

struct PADCCChunk {
    std::vector<TextureWeight> textureWeights;
};

```

PSBC — Parent sequence bounds:

```

struct PSBCChunk {
    std::vector<SequenceBounds> bounds; // Extent per parent sequence
};

```

PEDC — Parent event data:

```
struct PEDCChunk {
    std::vector<AnimationTrackBase> eventTracks;
};
```

13.6 LOD and Rendering Chunks

LDV1 — LOD configuration:

```
struct LDV1Chunk {
    u16 flags; // bit 3 (0x08) gates lodScaleRaw, see below
    u16 lodCount; // maxLod = lodCount - 1; client clamps to 8
    f32 lodDistance; // used in: fmaxf(fminf(740.0/lodDistance,
5.0), 0.5)
    std::array<u8, 4> particleBoneLod; // per-LOD particle bone mask
    u16 lodScaleRaw; // fixed point: scale = lodScaleRaw / 2048.0
    u8 lodBatchCount; // nonzero iff flags & 0x04
    u8 reserved1;
}; // 16 bytes (confirmed: all 3630 files = 16B)
```

Client behaviour (LDV1 is applied inline by the chunk loader, not deferred):

- **lodCount** is **clamped to 8**; the client also keeps **lodCount - 1** (or 0) as the max LOD index.
- **particleBoneLod** stores only **four** bytes but the client drives **eight** LOD slots: it copies **[0..3]** and then **replicates [3] into slots 4..7**.
- The two bytes at +12 are a single **u16** fixed-point scale: the client uses **lodScaleRaw / 2048.0** when **flags & 0x08** is set and **1.0** otherwise. Reading them as two separate bytes is what made the high byte look like a mirror of flags bit 3 — **0x0800 / 2048 == 1.0**.
- Two flag bits are set at runtime and are not file state: **0x04** when **lodDistance >= FLT_MIN**, and **0x8000** unconditionally.

LOD skins (e.g., **_lod01.skin**) are selected based on **entityLodDist** and **doodadLodDist** CVars. The **particleBoneLod** array determines particle visibility at each LOD:

```
// For LOD level L:
u32 mask = 0x10000 << particleBoneLod[L];
// For each particle emitter:
if (mask & bones[particle.boneIndex].flags) {
    // Suppress this emitter at this LOD
}
```

EDGF — Edge fade:

```
struct EDGFEntry {
    f32 fadeStart; // [0.0 .. 1.0] range – edge fade start distance factor
    f32 fadeEnd; // [0.0 .. 1.0] range – edge fade end distance factor
};
```

```

    f32 fadeDistance;    // [0.0 .. 10.0] range – distance multiplier or falloff
    u32 submeshIndex;    // submesh/batch index reference (observed range 0-25)
    u32 reserved0;       // always 0
    u32 reserved1;       // always 0
};                       // 24 bytes per entry
// Chunk is padded to 16-byte alignment: size = ceil(count * 24 / 16) * 16
// e.g., 1 entry → 32B, 2 → 48B, 3 → 80B

```

Applied to meshes when `ShadowBatch.flags2 & 0x8` is set. 685 files in the corpus contain EDGF data.

NERF — Distance-based model alpha:

```

struct NERFChunk {
    f32 squaredFarDist;    // squared distance at which alpha = 0 (fade-out
complete)
    f32 squaredNearDist;   // squared distance at which alpha = 1 (fully visible)
    u32 reserved[2];       // always 0
};                         // 16 bytes total
// alpha = (squaredFarDist - squaredRadius) / (squaredFarDist - squaredNearDist)

```

This value multiplies the model instance's alpha, creating a fade-out effect based on distance. Only 1 file observed in corpus (values: 400.0 = 20² yards, 25.0 = 5² yards).

DETL — Detail light parameters:

```

struct DETLEntry {
    u16 flags;             // always 0 in corpus
    f16 scale;             // shadow RT matrix scale (corpus: constant
0x231C ≈ 0.00062)
    f16 diffuseColorMultiplier; // multiplier for M2Light.diffuse_color (corpus:
constant 0x3C00 = 1.0)
    u16 unk0;              // always 0 in corpus
    u32 unk1;              // always 0 in corpus
};                         // 12 bytes per entry
// count = header.lights.count
// Chunk padded to 16-byte alignment: 1 light → 16B, 2 lights → 32B

```

278 files in the corpus contain DETL data. All observed entries have identical constant values across the entire corpus (115 entries, 100 sampled files).

13.7 Waterfall / PBR Chunks

These chunks signal that the model uses a separate "waterfall" render path with PBR-ish rendering and normal maps. Despite the name, this technology is used for many non-waterfall models (Shadowlands environment dressing, etc.).

WFFV1 / WFFV2 — Marker chunks. Empty payload structures in WhiteoutLib.

WV3 — Waterfall data v3:

```
struct WV3Data {
    f32 bumpScale;           // passed to vertex shader
    f32 value0_x, value0_y, value0_z;
    f32 value1_w, value0_w;
    f32 value1_x, value1_y;
    f32 value2_w, value3_y, value3_x;
    u8  baseColor[4];        // RGBA (not BGRA!)
    u16 flags;
    u16 unk0;
    f32 values3_w, values3_z, values4_y;
    f32 unk1, unk2, unk3, unk4;
};
```

The "value" fields are passed directly to the fragment shader.

13.8 Collision and Housing Chunks

PCOL — Player housing collision mesh (11.1.7+):

```
struct PCOLChunk {
    // Header:
    i32 vertexPosCount, vertexPosOffset;
    i32 faceNormCount, faceNormOffset;
    i32 indexCount, indexOffset;
    i32 flagsCount, flagsOffset;
    // Data (at respective offsets from chunk start):
    Vector3f vertexPositions[vertexPosCount];
    Vector3f faceNormals[faceNormCount];
    u16      indices[indexCount];
    u16      flags[flagsCount];
};
```

Use the offsets (relative to chunk data start) to seek — extra padding bytes may exist between sections.

DPIV — Pivot displacement data:

```
struct DPIVEntry {
    f32 offsetX;           // small displacement, often -0.0 or tiny negative
    f32 offsetY;           // small displacement, usually 0
    f32 offsetZ;           // varies significantly (-4.0 to +3.0 range)
    u32 flags;             // 0 or 1
    u32 reserved[4];       // always 0
};                          // 32 bytes per entry

struct DPIVChunk {
    DPIVEntry entries[]; // typically 1, observed up to 2
};
```

```
// Total size: N × 32 bytes. 14 files have 32 bytes (1 entry), 1 file has 64 bytes (2 entries).
```

15 files in the corpus contain DPIV data. The record count is confirmed by the client, which stores the payload pointer and `chunkSize / 32` as a separate count field and exposes that count directly — a reader that assumes a single 32-byte record silently drops the second one.

13.9 Miscellaneous Chunks

DBOC — Distance-based opacity control (seen in SL+):

```
struct DBOCEntry {
    f32 distanceOrScale; // range [0.001..10.0]; most common value ≈ 1.333
    (0x3FAAAAAB)
    f32 multiplier;      // range [0.0..13.0]; most common values: 1.5, 1.0
    u32 submeshIndex;    // submesh/batch reference (observed range 0–25)
    u32 reserved;        // always 0
};                       // 16 bytes per entry
// count varies: 1–12 entries observed per file
// Chunk size = count × 16 (always 16-byte aligned naturally)
```

1022 files in the corpus contain DBOC data. Size range: 16–512 bytes (1–32 entries). The `submeshIndex` values form a sequential pattern and appear to track which submesh each opacity control applies to.

AFRA — Animation frame rate (DF+):

```
struct AFRAChunk {
    std::vector<u8> data;
};
```

TEXL — Textured lights (12.0.0+):

```
struct TEXLEntry {
    f32 scaleU;          // UV scale U (corpus: always 1.0)
    f32 scaleV;          // UV scale V (corpus: always 1.0)
    i32 textureLookup;   // index in TXID for light cookie, or -1 (0xFFFFFFFF) if
    none
    i32 unk2;            // always 0 in corpus
};
// count = header.lights.count
```

Only 3 files in the corpus contain TEXL data. All observed entries are identical: `{1.0, 1.0, -1, 0}`.

14. Skeleton File (`.skel`)

Skeleton files allow bone/attachment/sequence data to be shared across models via a parent-child hierarchy. The file is always chunked. When a model references a `.skel` via the `SKID` chunk, the skeleton data overrides or supplements the M2 header's bone/attachment/sequence arrays.

```
struct SkeletonFile {
    std::optional<SKL1Chunk> skl1_chunk;
    std::optional<SKA1Chunk> ska1_chunk;
    std::optional<SKB1Chunk> skb1_chunk;
    std::optional<SKS1Chunk> sks1_chunk;
    std::optional<SKPDChunk> skpd_chunk;
    std::optional<AFIDChunk> afid_chunk;    // forwarded anim IDs
    std::optional<BFIDChunk> bfid_chunk;    // forwarded bone IDs
};
```

SKL1 — Skeleton header:

```
struct SKL1Chunk {
    u32 flags;                // 0x100 seen in Legion
    std::string name;
    std::array<u8, 4> reserved;
};
```

SKA1 — Skeleton attachments:

```
struct SKA1Chunk {
    std::vector<Attachment> attachments;
    std::vector<u16> attachmentLookupTable;
};
```

SKB1 — Skeleton bones:

```
struct SKB1Chunk {
    std::vector<Bone> bones;
    std::vector<u16> keyBoneLookup;
};
```

SKS1 — Skeleton sequences:

```
struct SKS1Chunk {
    std::vector<GlobalSequence> globalLoops;
    std::vector<Sequence> sequences;
    std::vector<u16> sequenceLookups;
```

```
std::array<u8, 8> reserved;
};
```

SKPD — Parent skeleton reference:

```
struct SKPDChunk {
    std::array<u8, 8> reserved0;
    u32 parentSkeletonFileId;    // fileDataId of parent .skel
    std::array<u8, 4> reserved1;
};
```

15. Bone File (.bone)

A **.bone** carries the skeleton edits one character-customization choice needs: a sparse set of replacement transforms, keyed by bone. A model ships one per choice — 2309 files across 84 models in the corpus, under **character/** (1276) and **creature/** (1033).

```
struct BoneFile {
    u32 version;                // 1 in every known file

    // BIDA – the bones this file touches
    u16 boneIndices[n];        // indexes the model's bone array

    // BOMT – one transform each
    f32 matrices[n][4][4];    // row-major, translation in the last row
};
```

Two chunks, in that order, with the ordinary **tag + u32 size** framing and tags stored forward (**BIDA**, **BOMT** — not reversed the way **.phys** stores them). There is no padding between chunks and no trailing bytes.

Verified across all 2309 corpus files:

Invariant	Result
<code>version == 1</code>	2309 / 2309
exactly BIDA then BOMT , nothing else	2309 / 2309
<code>sizeof(BIDA)/2 == sizeof(BOMT)/64</code>	2309 / 2309
no trailing bytes after the last chunk	2309 / 2309
<code>boneIndices</code> strictly ascending, no duplicates	2309 / 2309
4th matrix column is <code>(0, 0, 0, 1)</code>	86174 / 86174
<code>max(boneIndex) < model bone count</code>	972 / 972 files checked against their model

The ascending order is what lets the client binary-search a bone; the count is **not** tied to the number of **FacePose** sequences, as previously documented here — it is simply how many bones that customization choice moves. Of the 86174 matrices, 40969 carry rotation or shear, 10708 are pure scale and 34497 are translation-only; translations are small, with a median magnitude of 0.0014 and a 99th percentile of 0.41.

15.1 How a model finds its **.bone** files

Source	Condition	Where it lands
<stem>_NN.bone siblings	path mode; probed from _00 until one is missing	Model::boneOverrides[N]
BFID file ids	CASC mode; BFID[i] is the i- th file	Model::boneOverrides[i] , ids kept in Model::boneFileIds

BFID lives on the **.skel** for character models and on the **.m2** for models without one. Its length matches the sibling count for 74 of the 84 corpus models; the other 10 list more ids than the extraction contains.

The suffix must be **all digits** — **bloodelffemale_hd_sdr_00.bone** belongs to **bloodelffemale_hd_sdr**, not to **bloodelffemale_hd**, even though it shares the prefix.

16. Animation File (**.anim**)

Animation files externalize keyframe data for low-priority sequences (e.g., emotes, one-shot animations) to allow lazy loading.

Pre-Legion (raw)

The file is a raw binary blob of timestamps and values referenced by the M2's animation tracks. The **.anim** file to load is determined by:

```
{model_filename_without_extension}{anim_id:04d}-{sub_anim_id:02d}.anim
```

A file is loaded if **(sequence.flags & 0x130) == 0**. Blizzard's exporter aligns blocks to 16-byte boundaries, but this is not required.

Legion+ (chunked)

If the M2's **ChunkedAnimFiles** flag (0x2000) is set, or the model has a **.skel** file, **.anim** files use chunked framing:

```
struct AnimProfile {
    std::optional<AFM2Chunk> afm2_chunk; // animation data
    std::optional<AFSACChunk> afsa_chunk; // attachment animation data
    std::optional<AFSBChunk> afsb_chunk; // bone animation data
    bool isChunked;
};

struct AnimFile {
```

```
    AnimProfile profile;
    u32 animId;      // from filename
    u32 variant;     // from filename
};
```

- AFM2** — Contains the same animation data as old `.anim` files. For `.skel`-based models, this is restricted to event data only.
- AFSA** — Attachment animation data. Only present when the model uses a `.skel` file.
- AFSB** — Bone animation data. Only present when the model uses a `.skel` file.
- Detection logic:** WhiteoutLib peeks the first 4 bytes. If they match `AFM2`, `AFSA`, or `AFSB`, the file is chunked; otherwise, the entire file is treated as raw animation data stored in `afm2_chunk.animationData`.

17. Physics File (`.phys`) and PFDC

Physics data defines ragdoll simulation: rigid bodies, collision shapes, and constraints. Standalone `.phys` files use the same internal chunked format as PFDC inline data. All chunks use **reversed 4-byte ASCII tags** (standard WoW format), so `PHYS` on disk is stored as bytes `53 59 48 50`.

17.1 File Structure Overview

A physics file is a sequence of named chunks, each with the standard `u32_reversed_tag + u32_size + u8[size]` framing. Every chunk but `PLYT` is a flat array of fixed-size records, so its entry count is `chunk_size / stride`.

All 34 chunk orderings observed across the corpus are consistent with one sequence:

```
PHYS → PHYT → [BOXS] → [SPHS] → [CAPS] → [PLYT] → SHAP|SHP2 → BODY|BDY2|BDY3|BDY4
      → SHOJ|SHJ2 → WELJ|WLJ2|WLJ3 → [SPHJ] → [PRSJ|PRS2] → [REVJ|REV2] → [DSTJ] →
JOIN → [PHYV]
```

`BOXS`, `SPHJ`, `DSTJ` and `PRSJ` appear nowhere in the corpus, so their positions above are inferred from their neighbours. A file carrying `PHYV` carries nothing else but `PHYS` and `PHYT`.

Trailing padding. A standalone `.phys` ends at its last chunk. An inline PFDC payload is zero-padded to a **16-byte multiple** — 219 of 219 corpus payloads, including the 11 already aligned, which carry none. Read as chunk framing that padding looks like a null sentinel (`tag=0, size=0`), which is how it was first described; the alignment rule accounts for every case, the sentinel reading does not (observed pad lengths run 1–14 bytes, not just 0 and 8).

17.2 Version History

Version	Introduced	Body	Shape	Weld	Shoulder	Revolute	Notes
0	MoP 5.0.1.15464	BODY	SHAP	WELJ	—	—	

Version	Introduced	Body	Shape	Weld	Shoulder	Revolute	Notes
1	Legion 7.0.1.20773	BODY	SHAP	WELJ	—	—	Added PHYT
2	Legion 7.0.1.20979	BDY2	SHP2	WLJ2	—	—	Renamed chunks, added fields
3	Legion 7.0.3.21287	BDY3	SHP2	WLJ2	—	—	Added PLYT polytope shapes
4	Legion 7.0.3.21846	BDY4	SHP2	WLJ2	SHOJ	REVJ	
5	Legion 7.3.0.24500	BDY4	SHP2	WLJ2	SHOJ	REVJ	Parsing unchanged from 4
6	≥7.3, ≤9.0	BDY4	SHP2	WLJ3	SHJ2	REV2	Also PRS2 (prismatic)

Version 6 is what the wiki leaves open as `#if version >= ??` for the `SHJ2`/`WLJ3`/`REV2`/`PRS2` spellings. The corpus settles it: those four names occur **only** at version 6, and `SHOJ`/`WLJ2`/`REVJ` occur only at 3–5, with every chunk length an exact multiple of the corresponding stride.

Corpus version distribution (315 payloads: 96 standalone `.phys` + 219 inline PFDC):

Version	Standalone	PFDC
0	1	0
1	2	0
3	2	0
4	29	0
5	62	0
6	0	219

Chunk occurrence by version (files carrying each chunk):

Chunk	v0	v1	v3	v4	v5	v6
BODY / BDY3 / BDY4	1	2	1 (BDY3)	28	62	219
SHAP / SHP2	1	2	1	28	62	214
CAPS	1	2	1	28	61	199
SPHS	—	—	—	—	2	1
PLYT	—	—	—	24	31	85
WELJ / WLJ2 / WLJ3	1	2	1	15	19	39
SHOJ / SHJ2	—	—	1	27	58	181

Chunk	v0	v1	v3	v4	v5	v6
REVJ / REV2	—	—	1	27	31	67
PRS2	—	—	—	—	—	3
PHYV	—	—	1	1	—	—
JOIN	1	2	1	28	62	219

17.3 PHYS Header

```
struct PHYSHeader {
    u16 version;    // 0-6
};
```

Chunk size is always 2 bytes.

17.4 PHYT — Physics Type

```
struct PHYTEntry {
    u32 phyt;    // default 0
};
```

Chunk size is always 4 bytes. Observed values by version:

Version	Values
1	0 (2×)
3	2, 3
4	3 (26×), 0, 2, 4
5	4 (36×), 3 (21×), 0 (4×), 1
6	4 (181×), 3 (34×), 0 (4×)

Likely meaning: ragdoll complexity or physics simulation type (0=none/simple, 3=standard, 4=enhanced).

17.5 Body Structures

BODY/BDY2 keep the bone index at **+0x10** and address shapes with a 32-bit base; BDY3 moved the bone index up into the padding at **+0x02**, shrank the shape base to 16 bits, and appended the tuning floats.

```
struct BODYEntry {    // 28 bytes – versions 0-1
    u16 type;          // PhysicsBodyType, see below
    u16 padding_02;
    Vector3f position; // offset from the animated bone position
};
```

```

    u16 boneIndex;
    u16 padding_12;
    i32 shapeIndex;      // first entry in SHAP/SHP2 belonging to this body
    i32 shapeCount;
};

struct BDY2Entry {    // 32 bytes – version 2
    // ... BODYEntry ...
    f32 inertiaScale;  // dmBodyDef::m_inertiaScale
};

struct BDY3Entry {    // 44 bytes – version 3
    u16 type;
    u16 boneIndex;
    Vector3f position;
    u16 shapeIndex;
    u16 padding_12;
    i32 shapeCount;
    f32 gravityScale;  // dmBodyDef::m_gravityScale
    f32 inertiaScale;  // dmBodyDef::m_inertiaScale
    f32 linearDamping; // dmBodyDef::m_linearDamping
    f32 angularDamping; // dmBodyDef::m_angularDamping
    f32 unk_28;        // unidentified
};

struct BDY4Entry {    // 48 bytes – versions 4–6
    // ... BDY3Entry ...
    u16 unk_2c;        // unidentified; reads like a bit field
    u16 padding_2e;    // 0 in every corpus body
};

```

type is inverted relative to Domino. `Physics::LegacyLoadPhysData` maps file 1 → `dmBody_SetType(0)` and file 0 → `dmBody_SetType(1)`, and fatals with "Unknown legacy body type: %d" on anything else. `dmBody::Create` gives `dmBodyType` 0 unit mass, and `CPhysicsBodyDef::IsKinematicBody()` is `m_type == 1`. So:

file type	dmBodyType	Role
0	1 (kinematic)	Animation-driven collider. Glued to its bone; the solver only reads it.
1	0 (dynamic)	Simulated. Its bone transform is written back every frame — this is the cloth.

`position` is an **offset**, not an absolute placement: `CPhysicsBodyDef::CreateInstance` spawns the body at `modelPos + m_position`, where `modelPos` is the animated bone position.

Field value distributions (3526 bodies: 1213 kinematic, 2313 dynamic):

Field	Kinematic bodies	Dynamic bodies
<code>gravityScale</code>	1.0 (1168×), 0.0 (27×), 2.0 (16×)	1.0 (1137×), 0.75 (677×), 0.5 (158×), 0.0 (156×), −0.1 (25×), 20 more

Field	Kinematic bodies	Dynamic bodies
<code>inertiaScale</code>	1.0 (1193×), 1.5/2/5 (6× each)	1.0 (2264×), 10.0 (34×), 2.0 (6×), 1.1 (5×)
<code>linearDamping</code>	0.0 (1196×), 1.0 (15×)	3.0 (865×), 1.0 (693×), 0.0 (278×), 5.0 (173×), 4.0 (118×)
<code>angularDamping</code>	0.0 (1173×), 5.0 (21×), 10.0 (19×)	5.0 (1076×), 10.0 (821×), 0.0 (119×), 6.0 (61×)
<code>unk_28</code>	0.5 (853×), 0.9 (152×), 0.01 (123×), 0.25 (41×)	0.5 (1133×), 0.01 (672×), 0.9 (216×), 0.1 (185×)

The first four names come from `dmBodyDef`, whose only float members are exactly `m_linearDamping`, `m_angularDamping`, `m_gravityScale` and `m_inertiaScale`. The kinematic/dynamic split is what assigns them: a kinematic body is not integrated, so its damping is left at 0 (1196 and 1173 of 1213) while its `gravityScale/inertiaScale` sit at the `dmBodyDef` default of 1.0. `unk_28` is set on both kinds with the same distribution, so it is *not* a rigid-body integration parameter and stays unnamed.

`unk_2c` (BDY4+) is a `u16` followed by a `u16` that is zero in all 3526 bodies. The low half is 0 in 1823, `0x8000` alone in 494, and otherwise a small value (1, 2, 3, 6, 7, 8, 15, ...) — bit-field shaped, but unidentified.

17.6 Shape Structures

```
enum ShapeType : u16 {
    Box = 0,          // → BOXS data
    Capsule = 1,      // → CAPS data
    Sphere = 2,       // → SPHS data
    Polytope = 3      // → PLYT data (version 3+)
};

struct SHAPEntry {    // 20 bytes – versions 0-1
    u16 shapeType;
    i16 shapeIndex;    // index into the shape-data chunk shapeType names
    u32 padding_04;     // 0 in all 3230 corpus shapes
    f32 friction;       // CPhysicsShapeDef::SetFriction →
    dmFixtureDef::m_friction
    f32 restitution;   // CPhysicsShapeDef::SetRestitution
    f32 density;        // CPhysicsShapeDef::SetDensity
};

struct SHP2Entry {    // 32 bytes – versions 2+
    // ... SHAPEntry ...
    f32 unk_14;         // unidentified float: only 0, 0.01, 0.8 and 1.0 occur
    f32 scale;          // 1.0 in 3229 of 3230 – dmFixtureDef::m_scaleOrRadius
    u16 unk_1c;         // 0 in all 3230
    u16 padding_1e;     // uninitialized on disk; preserved verbatim
};
```

`CPhysicsShapeDef::InitDMFixtureDef` copies exactly `friction`, `restitution` and `density` into the `dmFixtureDef`, then overrides `m_filter.m_includeMask = 0` and sets `m_filter.m_groupIndex` to a per-model-instance counter — that group index is what keeps two players' capes from colliding with each other.

`m_scaleOrRadius` is initialised to 1.0 for every shape kind the client creates, which is what `scale` matches; `m_rollingResistance` is the one `dmFixtureDef` float that nothing else in the record accounts for, making it the candidate for `unk_14`, but with only 12 non-zero samples that stays a guess.

Shape kind distribution, over the 3038 shapes reachable from a body in the 289 corpus models that carry physics: Capsule = 2190, Polytope = 844, Sphere = 4, Box = 0. No corpus file uses `BOXS`.

Physics material values:

Property	Most common values
friction	0.6, 0.5, 0.7, 0.2
restitution	0.0, 0.1, 0.5
density	20000 (capsules), 100–29326 (polytopes vary)

17.7 Shape Data: CAPS (Capsule)

```
struct CAPSEntry {    // 28 bytes
    Vector3f localPosition1;
    Vector3f localPosition2;
    f32 radius;
};
```

Chunk size = `entry_count` × 28.

`CPhysicsCapsuleShapeDef::SetLocalPosition1/SetLocalPosition2/SetRadius` name all three fields.

`SPHS` is the same idea with one point (16 bytes: `Vector3f localPosition; f32 radius`), and `BOXS` is 60 bytes: a `PhysicsFrame` followed by `Vector3f halfExtents`. The client does not keep a box shape — `CPhysicsBoxShapeDef::SetPolytopeData(dmTransform, halfExtents)` converts it into a polytope immediately, which is why `dmFixtureDef::m_shapeType` is 2 (polytope) for both `BOXS` and `PLYT`.

PhysicsFrame

`BOXS` and every frame-based joint store the same twelve floats:

```
struct PhysicsFrame {    // 48 bytes
    Vector3f axisX;        // dmMtx::m_col[0]
    Vector3f axisY;        // dmMtx::m_col[1]
    Vector3f axisZ;        // dmMtx::m_col[2]
    Vector3f origin;       // dmTransform::p
};
```

`Physics::LegacyLoadPhysData` loads floats 0–8 as the three columns of a `dmMtx`, converts them with `dmQuatFromMtx`, and pairs the quaternion with floats 9–11 to make a `dmTransform`. It is a column-major 3×4 affine frame, not an opaque blob.

17.8 Shape Data: PLYT (Polytope) — CORRECTED

The PLYT chunk layout differs significantly from the C++ struct definitions. The chunk uses a "headers-then-data" layout, NOT interleaved header+data per entry.

PLYT chunk layout:

u32 entryCount	4 bytes
PLYTHeader[0]	80 bytes each
PLYTHeader[1]	
...	
PLYTHeader[entryCount-1]	
PLYTData[0]	variable size each
PLYTData[1]	
...	
PLYTData[entryCount-1]	

The header is a serialised **dmPolytope**: each count sits in front of its own 64-bit pointer, leaving a four-byte hole that the exporter never initialises.

```

struct PLYTHeader {    // 80 bytes (0x50)
    u32 vertexCount;    // +0x00
    u32 padding_04;     // +0x04: uninitialised
    u64 runtime_vertices; // +0x08: 0 on disk, dmPolytope::m_pVertices at runtime
    u32 faceCount;      // +0x10
    u32 padding_14;     // +0x14: uninitialised
    u64 runtime_planes;  // +0x18: 0 on disk, dmPolytope::m_pPlanes
    u64 runtime_faces;   // +0x20: 0 on disk, dmPolytope::m_pFaceEdges
    u32 edgeCount;      // +0x28: half-edges, always even
    u32 padding_2c;     // +0x2C: uninitialised
    u64 runtime_edges;   // +0x30: 0 on disk, dmPolytope::m_pEdges
    Vector3f centroid;   // +0x38: volume-weighted
    f32 volume;          // +0x44
    f32 surfaceArea;     // +0x48
    u32 padding_4c;     // +0x4C: uninitialised
};

struct PLYTData {
    Vector3f vertices[header.vertexCount];
    Vector4f facePlanes[header.faceCount]; // (nx, ny, nz, d)
    u8 faceFirstEdge[header.faceCount];    // any half-edge bounding that face
    dmSubEdge edges[header.edgeCount];
};

struct dmSubEdge {    // 4 bytes
    i8 twinOffset;    // twin of edge i is (i + twinOffset); only +1 and -1 occur
    u8 originVertex;  // index into vertices
};

```

```

    u8 faceIndex;    // index into facePlanes
    u8 nextEdge;     // next half-edge around faceIndex
};

```

Data size per entry: $\text{vertexCount} \times 12 + \text{faceCount} \times 16 + \text{faceCount} + \text{edgeCount} \times 4$

This is **not** a BSP tree — it is Domino's half-edge mesh, and the client's two accessors define the byte roles exactly. `dmPolytope::FindEdge(v1, v2)` matches `edges[i].originVertex == v1` and then walks `edges[i + edges[i].twinOffset].originVertex == v2`, so byte 0 is a signed step to the twin and byte 1 is the origin vertex. `dmPolytope::GetSupportFace` starts at `m_pFaceEdges[face]` and loops on `edge = edges[edge].nextEdge` until it returns to the start, so byte 3 is the next edge around the face and the per-face byte array is that face's entry point.

Corpus verification (875 hulls, 6031 faces, 24712 half-edges — every one of them):

Invariant	Result
<code>twinOffset</code> is ± 1 , and <code>twin(twin(e)) == e</code>	24712 / 24712
<code>origin(twin(e)) == origin(next(e))</code> — the edge's other end	24712 / 24712
<code>faceFirstEdge[f]</code> is an edge whose <code>faceIndex == f</code>	6031 / 6031
the <code>nextEdge</code> cycle closes and every edge in it carries <code>faceIndex == f</code>	6031 / 6031
every half-edge is reached by exactly one face walk	24712 / 24712
<code>volume</code> and <code>surfaceArea</code> positive and within the vertex AABB's	875 / 875
<code>centroid</code> equals the vertex mean	847 / 875 (rest are volume-weighted)
all four runtime pointers zero on disk	875 / 875

Face cycle lengths run 3–16, mostly quads (5360 of 6031). Hulls carry 8–77 vertices; the common 8-vertex / 6-face / 24-half-edge case is a box. The $+1/-1$ split of `twinOffset` is exactly even (12356 each), which is what "half-edges are stored in twin pairs at adjacent indices" means in practice.

17.9 Joint Structure

```

struct JOINEntry {    // 16 bytes
    u32 bodyAIdx;      // CPhysicsJointDef::SetBodyAIdx
    u32 bodyBIdx;      // CPhysicsJointDef::SetBodyBIdx
    u32 padding_08;    // 0 in all 2449 corpus joints
    u16 jointType;      // JointType enum
    i16 jointId;        // joint sub-index within its type chunk
};

```

The `jointType` values are the `CPhysicsDef::Create*Joint` ordering, **not** Domino's `dmJointType` (distance 1, mouse 2, prismatic 3, revolute 4, shoulder 5, spherical 6, weld 7):

file	jointType	chunk	client class
0		SPHJ	CPhysicsSphericalJointDef
1		SHOJ/SHJ2	CPhysicsShoulderJointDef
2		WELJ/WLJ2/WLJ3	CPhysicsWeldJointDef
3		REVJ/REV2	— (post-6.0.1)
4		PRSJ/PRS2	— (post-6.0.1)
5		DSTJ	— (post-6.0.1)

Joint type distribution across the 289 corpus models with physics: Shoulder = 1701, Revolute = 324, Weld = 240, Prismatic = 12. Shoulder joints are what chain cloth; welds mostly bolt the static collider capsules together.

17.10 Joint Types

Version mapping:

- v4–5: SHOJ + WLJ2 + REVJ
- v6: SHJ2 + WLJ3 + REV2

SHOJ (Shoulder Joint) — 116 bytes

```
struct SHOJEntry {
    PhysicsFrame frameA;           // 48 bytes – reference frame in body A
    PhysicsFrame frameB;           // 48 bytes – reference frame in body B
    f32 lowerTwistAngle;           // -15.0 (112×) or -20.0 (84×) degrees
    f32 upperTwistAngle;           // 15.0 (112×) or 20.0 (86×) degrees
    f32 coneAngle;                 // 45.0 (120×), 35.0 (60×), 20.0, 60.0 degrees
    f32 maxMotorTorque;            // 0.1 (157×), 1000.0 (35×), 0.01 (8×)
    u32 motorMode;                 // 0=free (35×), 1=motor (165×)
};
```

CPhysicsShoulderJointDef names the first three: `SetLowerTwistAngle`, `SetUpperTwistAngle`, `SetConeAngle`. The angles are stored in **degrees** — `dmShoulderJoint::Create` clamps its own cone to `[0.17453294, 2.9670596]`, i.e. `[10°, 170°]` in radians, so the loader converts on the way in. The default `dmJointDef` cone before the loader touches it is $\pi/4$.

SHJ2 (Shoulder Joint v2) — 124 bytes

Extends SHOJ with:

```
f32 motorFrequencyHz;           // 1.0 (48×), 1.5 (2×), 3.0 (2×)
f32 motorDampingRatio;          // 0.7 (39×) or 0.0 (13×)
```

WLJ2 (Weld Joint v2) — 112 bytes

```
struct WLJ2Entry {
    PhysicsFrame frameA;
    PhysicsFrame frameB;
    f32 angularFrequencyHz;    // always 0.6 in corpus
    f32 angularDampingRatio;    // always 0.0
    f32 linearFrequencyHz;      // always 0.0
    f32 linearDampingRatio;     // always 0.0
};
```

WLJ (version 0–1, 104 bytes) has only the first pair. Which pair is which comes from the client: **CPhysicsWeldJointDef::SetFrequency/SetDampingRatio** write **dmJointDef+0x70/+0x78**, which **dmWeldJoint::Create** copies to the joint's first softness slot, and **InitVelocityConstraints** reads that slot to gate the block that composes **frameA.q**, **frameB.q** and both body quaternions — the orientation constraint. The second pair drives the point constraint further down. Zero frequency means that axis is solved rigidly, which is why every corpus weld leaves the linear pair at 0.

WLJ3 (Weld Joint v3) — 116 bytes

Extends WLJ2 with:

```
f32 unk70;                // 0.0 (265x), 0.005 (7x), 10.0, 5.0 – unidentified
```

REVJ (Revolute Joint) — 112 bytes

```
struct REVJEntry {
    PhysicsFrame frameA;
    PhysicsFrame frameB;
    f32 lowerAngle;          // -60.0 (32x), -20.0, -30.0, -10.0 degrees
    f32 upperAngle;          // 60.0 (32x), 20.0, 30.0, 10.0 degrees
    f32 maxMotorTorque;       // 0.0 (16x), 0.1 (16x), 0.01 (5x)
    u32 motorMode;           // 0 (16x) or 1 (21x)
};
```

REV2 (Revolute Joint v2) — 120 bytes

Extends REVJ with:

```
f32 motorFrequencyHz;      // always 1.0 in corpus
f32 motorDampingRatio;     // always 0.7 in corpus
```

`dmRevoluteJoint::Create` reads `dmJointDef+0x70/+0x74` as the angle limits and `+0x78/+0x7C` as the motor pair, which is the order the file uses.

SPHJ (Spherical Joint) — 28 bytes

```
struct SPHJEntry {
    Vector3f anchorA;           // CPhysicsSphericalJointDef::SetAnchorA
    Vector3f anchorB;           // SetAnchorB
    f32 frictionTorque;         // SetFictionTorque – Blizzard's own typo
};
```

DSTJ (Distance Joint) — 28 bytes

```
struct DSTJEntry {
    Vector3f localAnchorA;
    Vector3f localAnchorB;
    f32 distance;               // dmDistanceJoint::Create broadcasts it as the
    target length
};
```

PRSJ / PRS2 (Prismatic Joint) — 120 / 128 bytes

```
struct PRSJEntry {
    PhysicsFrame frameA;
    PhysicsFrame frameB;
    f32 lowerLimit;
    f32 upperLimit;
    f32 unk_68;                 // 0 in all twelve corpus prismatic joints
    f32 maxMotorForce;
    f32 unk_70;                 // 0 in all twelve
    u32 motorMode;
    // PRS2 only:
    f32 motorFrequencyHz;
    f32 motorDampingRatio;
};
```

`dmPrismaticJoint::Create` takes the limit pair, then a bool, then the motor force and speed, so the two unknowns are most likely the enable-limit / enable-motor flags — but every corpus value is 0, so nothing distinguishes them from padding.

Neither `SPHJ`, `DSTJ` nor `BOXS` occurs anywhere in the corpus; those three layouts are covered only by the synthetic write-then-read case in `m2_phys_test`.

17.11 PFDC Inline Physics

When physics is inlined in the `.m2` via the **PFDC** chunk (Shadowlands+), the payload is a complete `.phys` file — same chunks, same framing — zero-padded to a 16-byte multiple. The chunk size covers the padding.

All 219 PFDC payloads in the corpus are version 6, using the latest struct variants (BDY4, SHP2, SHJ2, WLJ3, REV2, PRS2).

WhiteoutLib parses the payload into `PhysicsData` and regenerates the padding on write; `m2_phys_test` checks that all 315 corpus payloads come back byte for byte.

17.12 How a model finds its physics

Source	Condition	Where it lands
<code><stem>.phys</code> sibling	MD20 with the <code>LoadPhysicsData</code> global flag, path mode	<code>Model::physics</code>
<code>PFID</code> file id	MD21 carrying <code>PFID</code> , resolved by id in CASC mode or by sibling path otherwise	<code>Model::physics</code> , id kept in <code>Model::physicsFileId</code>
<code>PFDC</code> chunk	MD21 carrying the payload inline	<code>Model::physics</code>

The writer puts it back the way it came: a model with `physicsFileId` set emits `PFID` and a separate `.phys`, one without emits `PFDC` (MD21) or a `.phys` sibling (MD20).

17.13 Where the field names come from

The 6.0.1 macOS client ships with unstripped C++ symbols for the whole physics path, which is what fixes the names above rather than guessing them from value distributions. The relevant entry points:

Symbol	What it settles
<code>PhysData::Load</code>	the chunk table: which tag feeds which array, and each stride (<code>BOXS</code> /60, <code>CAPS</code> /28, <code>SPHS</code> /16, <code>SHAP</code> /20, <code>BODY</code> /28, <code>SPHJ</code> /28, <code>SHOJ</code> /108, <code>WELJ</code> /104, <code>JOIN</code> /16)
<code>Physics::LegacyLoadPhysData</code>	the <code>type</code> inversion, the frame → <code>dmTransform</code> conversion, and the field order within every version-0 record
<code>CPhysicsBodyDef</code> / <code>CPhysicsShapeDef</code> / <code>CPhysics*JointDef</code> setters	the individual field names
<code>CPhysicsBodyDef::CreateInstance</code>	<code>position</code> is an offset from the animated bone position
<code>CPhysicsShapeDef::InitDMFixtureDef</code>	friction/restitution/density mapping and the per-instance collision filter
<code>dmPolytope::FindEdge</code> / <code>GetSupportFace</code>	the four bytes of a <code>PLYT</code> half-edge
<code>dmBodyDef</code> , <code>dmFixtureDef</code> , <code>dmPolytope</code> , <code>dmFilter</code>	the candidate names for fields no 6.0.1 code path reads

6.0.1 only accepts **PHYS** version 0 (**PhysData::Load** bails unless the version word is zero), so everything version 2 and later — **PLYT**, the **BDY3** tuning floats, the **SHP2** tail, the motor parameters, **PHYT** and **PHYV** — is named from the Domino structures plus corpus evidence, not from a symbol the loader reads. Sections above say which is which.

18. Chunked Geometry and Model3 — 11.x Client Formats

Status: identified by reverse-engineering the 11.x retail client. These chunk families are **not implemented by WhiteoutLib** and have **not** been corpus-validated — unlike the rest of this document, the tag lists below come from the client's dispatch code alone. Payload layouts are given only where the client's own code shows them.

18.1 Chunked Geometry (**.skin** successor)

Modern builds carry a second, chunked geometry container alongside the classic **.skin** profile of [Section 12](#). Each vertex stream is a chunk whose payload begins with a **format code** — itself a FourCC — followed by the packed elements:

```
struct GeoStreamChunk {
    u32 magic;      // e.g. 'VPOS'
    u32 size;       // payload bytes, excluding this 8-byte header
    u32 format;     // M2VertexFormat, see below
    u8  data[];    // size - 4 bytes of packed elements
};
```

Format	Meaning	Bytes/element
1U16	1 × u16	2
2F32	2 × f32	8
3F32	3 × f32	12
4U8N	4 × u8, normalised	4

Chunk	Contents	Observed format
M3VR	Container version	—
VGEO	Geometry descriptor	—
RBAT	Render batches	—
LODS	LOD table	—
VPOS	Vertex positions	3F32
VNML	Vertex normals	3F32
VTAN	Vertex tangents	—
VUV0–VUV5	UV sets 0–5	2F32

Chunk	Contents	Observed format
VCL0, VCL1	Vertex colours	4U8N
VWTS	Vertex weights	—
VIBP	Bone indices / bind pose	—
VSTR	Vertex stream descriptor	—
VINX	Vertex indices	1U16
SKIN	Skin profile container	—

The client can also synthesise one of these buffers procedurally: a 1380-byte buffer describing a **unit box** (24 vertices via **VPOS/VNML** at 288 B each, 24 UVs per set at 192 B, 24 colours at 96 B, and 36 indices at 72 B).

18.2 Geometry Collision Chunks

A separate collision triple appears in the same family:

Chunk	Contents
CPOS	Collision vertex positions
CNML	Collision face normals
CINX	Collision indices

18.3 Model3 (.m3) — WoW

11.x also contains a parser for a model container tagged **M3***. **This is not the StarCraft II / Heroes of the Storm MD34 format** described in **M3_FILE_FORMAT_SPECIFICATION.md**; the two share only the name.

Structural chunks: **M3DT**, **M3CL**, **M3EP**, **M3SI**, **M3ST**, **M3VS**, **M3XF**, **M3PT**, **MES3**.

Material chunks are stored with **byte-reversed** FourCCs, the same convention **.phys** uses (see [Section 17](#)) — the bytes on disk are the reverse of the name below:

Name	On-disk bytes
MAIN	NIAM
MALI	ILAM
MALS	SLAM
EMIS	SIME
BURS	SRUB
SHDR	RDHS
P3DT	TD3P

Payload layouts for both families are still unknown.

19. Bundle Naming Conventions

WhiteoutLib's `collectBundle()` discovers sibling files by stem-matching in the same directory as the `.m2`:

File Type	Pattern	Example
Base model	<code>{stem}.m2</code>	<code>scorpion.m2</code>
Skeleton	<code>{stem}.skel</code>	<code>scorpion.skel</code>
Base skin	<code>{stem}{NN}.skin</code>	<code>scorpion00.skin</code> , <code>scorpion01.skin</code>
LOD skin	<code>{stem}_lod{NN}.skin</code>	<code>scorpion_lod01.skin</code>
Animation	<code>{stem}{NNNN}-{NN}.anim</code>	<code>scorpion0069-01.anim</code>
Bone	<code>{stem}_{NN}.bone</code>	<code>scorpion_07.bone</code>

Where `{NN}` is zero-padded to 2 digits and `{NNNN}` to 4 digits.

`fromFileSystem()` performs the reverse: given an in-memory `FileSystem` and an output path, it computes all output file paths using the same conventions. Falls back to `"just_another_model"` if the base name is empty.

20. WhiteoutLib Implementation Notes

20.1 Parser

```
whiteout::m2::Parser parser(ParseMode::Lenient);
FileSystem fs = parser.parse("path/to/model.m2");
```

- **ParseMode::Strict**: throws on errors or unknown chunks
- **ParseMode::Lenient**: accumulates issues and continues; retrievable via `parser.getIssues()`

Parse order for bundles:

1. Base `.m2` (detect MD20 vs MD21 by first 4 bytes)
2. Base skins (sorted by index)
3. LOD skins (sorted by LOD level)
4. `.skel` skeleton
5. `.bone` files (sorted by bone ID)
6. `.anim` files (sorted by anim ID, then variant)

Magic detection:

- `0x4D443230` ("MD20") → flat classic parser
- `0x4D443231` ("MD21") → chunked parser
- `0x534B494E` ("SKIN") → skin parser

Offset resolution: The `BinaryParseVisitor` records `baseOffset` at the start of parsing. For `ClassicMD20`, `baseOffset = 0`. For `MD21`, `baseOffset =` position after the chunk header. All `M2Array` offsets resolve as `offset + baseOffset`.

Chunk parsing: Order-independent `while(hasRemaining())` loop with `switch(tag)`. After each handler, the reader seeks to `chunkStart + chunkSize` to guarantee forward progress even on partial reads.

Unknown chunks: Reported via the issue system and skipped by their declared size.

20.2 Writer

```
whiteout::m2::Writer writer;
writer.write("path/to/output.m2", fs);
```

- Writes base `.m2`, all skins, and `.skel` (if present) to disk
- `Writer::write(const BaseFile&)` returns `std::vector<u8>` for in-memory serialization
- Pre-allocates 2 MB buffer, then `shrink_to_fit()`

Chunk write order (deterministic): `MD21` → `LDV1` → `PFID` → `SFID` → `AFID` → `BFID` → `TXAC` → `EXPT` → `EXP2` → `PABC` → `PADC` → `PSBC` → `PEDC` → `SKID` → `TXID` → `RPID` → `GPID` → `WV1` → `WV2` → `PGD1` → `WV3` → `PFDC` → `EDGF` → `NERF` → `DETL` → `DBOC` → `AFRA` → `PCOL` → `DPIV` → `TEXL`

Chunks are written back-to-back with no inter-chunk alignment padding. Each chunk writes: tag (4B) → size placeholder (4B) → payload → seek back to patch actual size → return to end.

PFDC: Serialized from `PhysicsData` by `m2::writePhysics()`, then zero-padded to a 16-byte multiple.

20.3 API Surface

Public headers:

Header	Purpose
<code>include/whiteout/m2/m2.h</code>	Umbrella include
<code>include/whiteout/m2/types.h</code>	Core types, version constants, tags
<code>include/whiteout/m2/structures.h</code>	Top-level <code>FileSystem</code> , <code>BaseFile</code> , <code>SkinFile</code> , etc.
<code>include/whiteout/m2/structures/base.h</code>	<code>MD20Header</code> and all contained structures
<code>include/whiteout/m2/structures/skin.h</code>	Skin profile, sections, batches
<code>include/whiteout/m2/structures/chunks.h</code>	All 30 chunk type definitions
<code>include/whiteout/m2/structures/skeleton.h</code>	Skeleton chunks
<code>include/whiteout/m2/structures/bone.h</code>	Bone file structures
<code>include/whiteout/m2/structures/anim.h</code>	Animation file structures
<code>include/whiteout/m2/structures/phys.h</code>	Physics structures (<code>PhysicsData</code> and its records)

Header	Purpose
<code>include/whiteout/m2/phys_file.h</code>	<code>parsePhysics()</code> / <code>writePhysics()</code> for standalone <code>.phys</code>
<code>include/whiteout/m2/parser.h</code>	Parser class
<code>include/whiteout/m2/writer.h</code>	Writer class

Top-level in-memory model:

```
struct FileSystem {
    std::string baseName;
    BaseFile base;
    std::vector<SkinFile> skins;
    std::vector<AnimFile> anims;
    std::vector<BoneFile> bones;
    std::optional<SkeletonFile> skeleton;
    std::optional<PhysicsFile> physics;
};
```

Typical usage:

```
#include <whiteout/m2/m2.h>

// Parse
whiteout::m2::Parser parser(whiteout::m2::ParseMode::Lenient);
whiteout::m2::FileSystem model = parser.parse("creature/scorpion/scorpion.m2");

// Access issues
for (const auto& issue : parser.getIssues()) {
    std::cerr << issue << "\n";
}

// Inspect header
auto& hdr = model.base.header;
std::cout << "Version: " << hdr.version << "\n";
std::cout << "Bones: " << hdr.bones.size() << "\n";
std::cout << "Vertices: " << hdr.vertices.size() << "\n";
std::cout << "Sequences: " << hdr.sequences.size() << "\n";

// Round-trip write
whiteout::m2::Writer writer;
writer.write("output/scorpion.m2", model);
```

Example programs:

- `examples/m2_loader_example.cpp` — Loads and prints model statistics
- `examples/m2_writer_example.cpp` — Loads, then writes to output path (round-trip)

20.4 Known Limitations

Area	Status
AFRA chunk	Stored as raw <code>vector<u8></code>
Pre-WotLK skins	Inline skin profiles not implemented (skins must be external <code>.skin</code> files)
Pre-WotLK tracks	Single-timeline animation blocks (with <code>interpolation_ranges</code>) not implemented
Anim file writing	<code>Writer::write(path, model)</code> emits base, skins, skeleton, <code>.phys</code> and <code>.bone</code> ; <code>.anim</code> siblings only when the model splits them
PHYS chunks not in the corpus	<code>BOXS</code> , <code>SPHJ</code> , <code>DSTJ</code> and <code>PRSJ</code> are implemented from the wiki but unverified; so is version 2, of which no file was found
Vanilla rotation format	Full <code>C4Quaternion</code> (16-byte) bone rotation (pre-BC) not separately handled
Chunked geometry	The <code>VPOS/VNML/VINX/...</code> stream container of Section 18 is not parsed
Model3 (<code>.m3</code> , WoW)	Chunk tags identified, no parser; distinct from the SC2/HotS <code>MD34</code> format
DPIV record fields	Record count and 32-byte stride confirmed; field meanings still inferred from corpus values

21. Corpus Validation Results

This specification was validated against a corpus of **9059 M2 files** and **14443 SKIN files** from `Models/WoW/` (covering Dragonflight through The War Within 11.x era models). Analysis scripts are in `scripts/m2_corpus_analysis.py`, `scripts/m2_pgd1_analysis.py`, and `scripts/m2_seqflags_analysis.py`.

21.1 Overview

Statistic	Value
Total M2 files	9059
Classic (MD20)	0 (all files are chunked MD21)
Parse errors	0
SKIN files	14443
SKIN parse errors	0
Version 272 (1.16)	3629 files
Version 274 (1.18)	5430 files
Unique chunk tags	26 of 30 documented
Unknown chunk tags	0

21.2 Key Findings

Global Flags (Section 6.2):

- `0x80` is set on **all 9059 files** — universal in modern exports, not DH-tattoo-specific
- `0x10` appears in **5664 files** (62%) — previously undocumented, very common
- `0x200000` appears in **4274 files** (47%) — previously undocumented, very common
- `0x100` (CameraRelated) appears in only **3 files** — contradicts prior claim of "every model since Cata"

Sequence Flags (Section 7.3):

- 45310 total sequences across 9059 files (avg ~5/file)
- 10919 unique flag values
- Bit 0 (`0x01`) used in 19220 sequences — previously undocumented
- `0x7FFF` (all bits 0–14) is the second-most common value (6347 sequences), typically for aliases
- Upper 16 bits non-zero in ~31% of sequences; when set, lower 16 bits are almost always `0xFFFF`

Material Blend Modes (Section 8.4):

- Modes 5, 6, 7 discovered; mode 7 is **very common** (4133 materials)
- Mode 3 not observed in this corpus

Texture Types (Section 8.3):

- Types 21–24 not observed in the corpus
- Types 25 (4 files) and 26 (2 files) confirmed

PGD1 (Section 13.4):

- Completely re-documented: M2Array-style header (16 bytes) + u16 geoset data, not flat u16 per particle
- Size formula: $16 + \text{ceil}(\text{count} \times 2 / 16) \times 16$ — matches all 2077 files exactly

EDGF (Section 13.6):

- Entry size confirmed as 24 bytes with 3 identified fields (2×f32 fade factors, 1×f32 distance, 1×u32 index, 2×u32 reserved)

DBOC (Section 13.9):

- Entry size confirmed as 16 bytes: {f32 distance, f32 multiplier, u32 index, u32 reserved}
- 1022 files, 1–32 entries per file

SKIN Batches:

- Shadow batch flags distribution: `0x10` (56698), `0x90` (7216), `0x80` (1406)
- Shadow batch flags2 distribution: `0x02` (42358), `0x03` (14511), `0x00` (5778)
- Regular batch flags: `0x10` (62561), `0x80` (22942), `0x88` (10715)

21.3 Size Validation

Chunk	Validation	Result
TXID	size = textures.count × 4	✓ All match
SFID	size = numSkinProfiles × 4 (min)	✓ All match
PGD1	size = 16 + align16(particles × 2)	✓ All 2077 files match
LDV1	always 16 bytes	✓ All 3630 files match
EDGF	size = align16(entries × 24)	✓ All sizes explained
DBOC	size = entries × 16	✓ All sizes explained
DETL	size = align16(lights × 12)	✓ All sizes explained

21.4 PHYS Corpus Validation

Validated against **96 standalone .phys files** and **219 inline PFDC chunks** (315 payloads) from [Corpus/WoW](#). The check is [tests/m2_phys_test.cpp](#): parse, write, compare bytes.

Statistic	Value
Standalone .phys files	96 (v0: 1, v1: 2, v3: 2, v4: 29, v5: 62)
Inline PFDC chunks	219 (all v6)
Parse errors	0
Unknown chunks	0
Byte-exact round-trips	315 / 315
PLYT entries	875
Polytope half-edges checked	24712, against five structural invariants each

Key PHYS findings:

- **PLYT layout:** headers-then-data with a **u32** count prefix, and within a payload the order is vertices → face planes (**Vector4f**) → one byte per face → half-edges. The edge block being last is what makes every **twinOffset** ±1 and every index land in range; any other split scatters them.
- **PLYT is a half-edge mesh, not a BSP tree.** **twinOffset/originVertex/faceIndex/nextEdge** all hold across 24712 of 24712 edges and 6031 of 6031 face cycles — see §17.8.
- **The u32s beside each PLYT count are struct padding**, not version-specific metadata: they hold uninitialised heap bytes, sometimes fragments of unrelated ASCII strings. Same for the sixth float of the header tail, which is an integer-valued denormal rather than a float.
- **Body damping is kinematic-gated:** **linearDamping** and **angularDamping** are 0 on 1196 and 1173 of the 1213 kinematic bodies and freely tuned on dynamic ones, which is what identifies them against **dmBodyDef**.
- **Version 6 chunk names:** **SHJ2** (0x7C), **WLJ3** (0x74), **REV2** (0x78) and **PRS2** (0x80) occur only at version 6, and **SHOJ/WLJ2/REVJ** only at 3–5. This is what the wiki leaves as **#if version >= ??**.
- **PFDC padding is 16-byte alignment**, not a fixed 6 bytes or an 8-byte null sentinel: observed pad lengths run 1–14 bytes and always bring the payload to a 16-byte multiple.

- **SHP2 unk_1e** is uninitialized padding (ASCII garbage in v4, zeroed in v5+), so it is preserved verbatim rather than normalized.
 - **All PLYT runtime pointer fields are zero on disk** (875/875 entries × 4 fields); the client fills them in after load, so they are not stored in **PolytopeShape**.
-

22. References

1. **wowdev.wiki M2**: <https://wowdev.wiki/M2> — Community reverse-engineering documentation
2. **wowdev.wiki M2/.skin**: <https://wowdev.wiki/M2/.skin> — Skin file format
3. **wowdev.wiki M2/.skel**: <https://wowdev.wiki/M2/.skel> — Skeleton file format
4. **wowdev.wiki PHYS**: <https://wowdev.wiki/PHYS> — Physics file format
5. **wowdev.wiki BONE**: <https://wowdev.wiki/BONE> — Bone file format
6. **wowdev.wiki M2/Rendering**: <https://wowdev.wiki/M2/Rendering> — Shader and blend mode details
7. **Warcraft III Art Tools**: <http://ftp.blizzard.com/pub/war3/other/WarcraftIIIArtTools1.01.zip> — Historical MDX/PRE2 particle flag reference